Lecture 2

CME 295: Transformers & Large Language Models





Afshine Amidi & Shervine Amidi

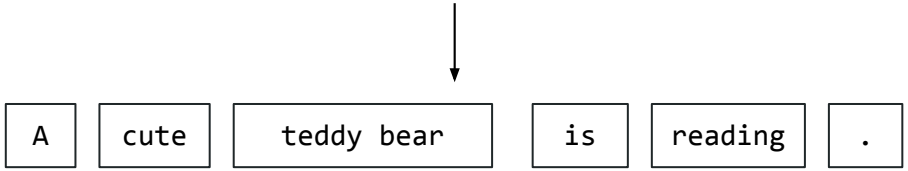


Stanford University

ICME

Recap of last episode...

A cute teddy bear is reading.



Recap of last episode...

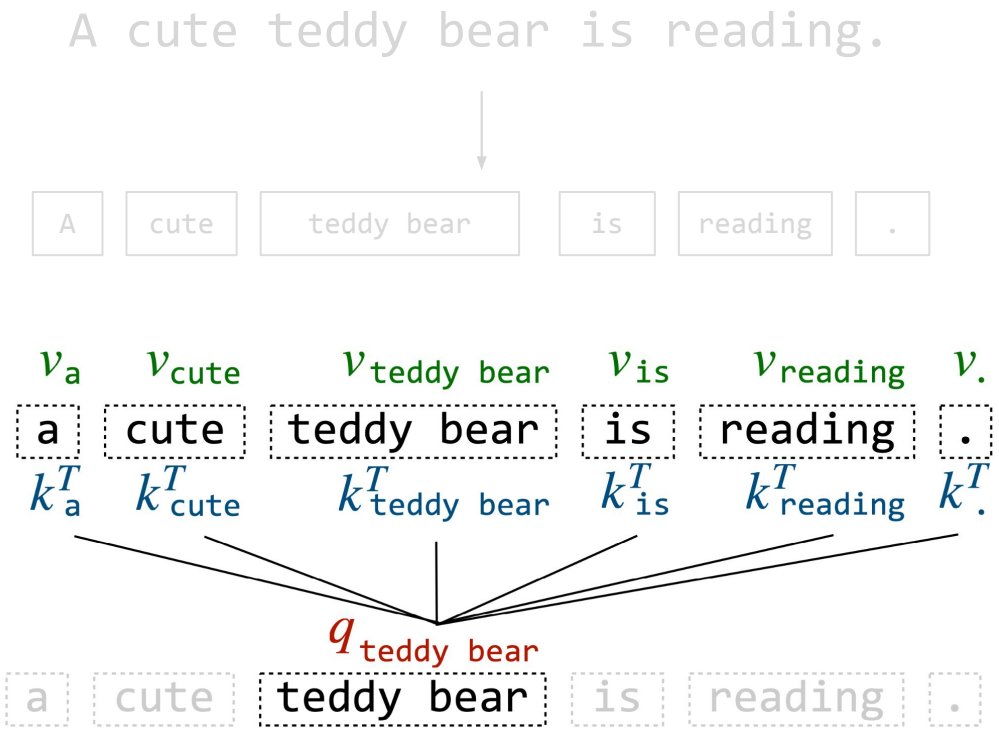
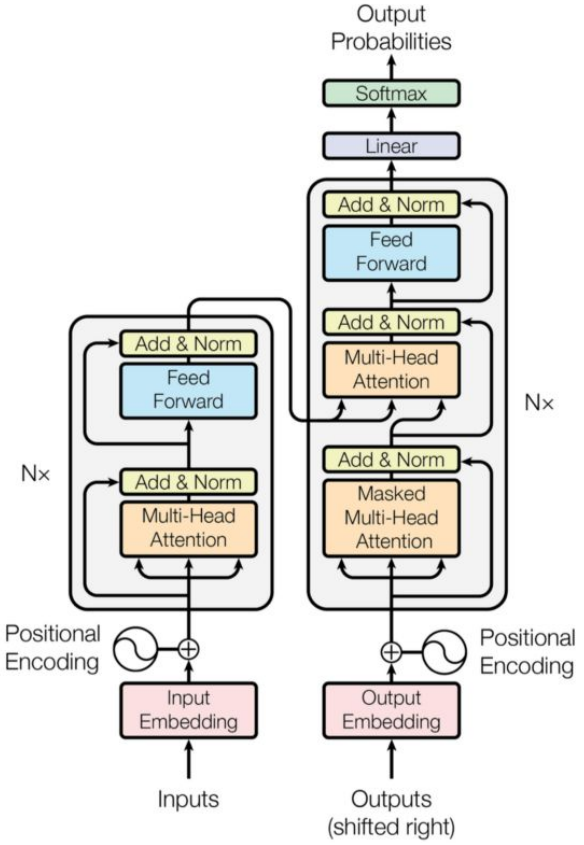


Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Recap of last episode...

A cute teddy bear is reading.



Attention Is All You Need, Vaswani et al., 2017.



Transformers & Large Language Models

Transformer model families

Scaling laws

Expert routing

Architecture optimizations

Decoding strategies

Prompting and reasoning

Transformer architecture

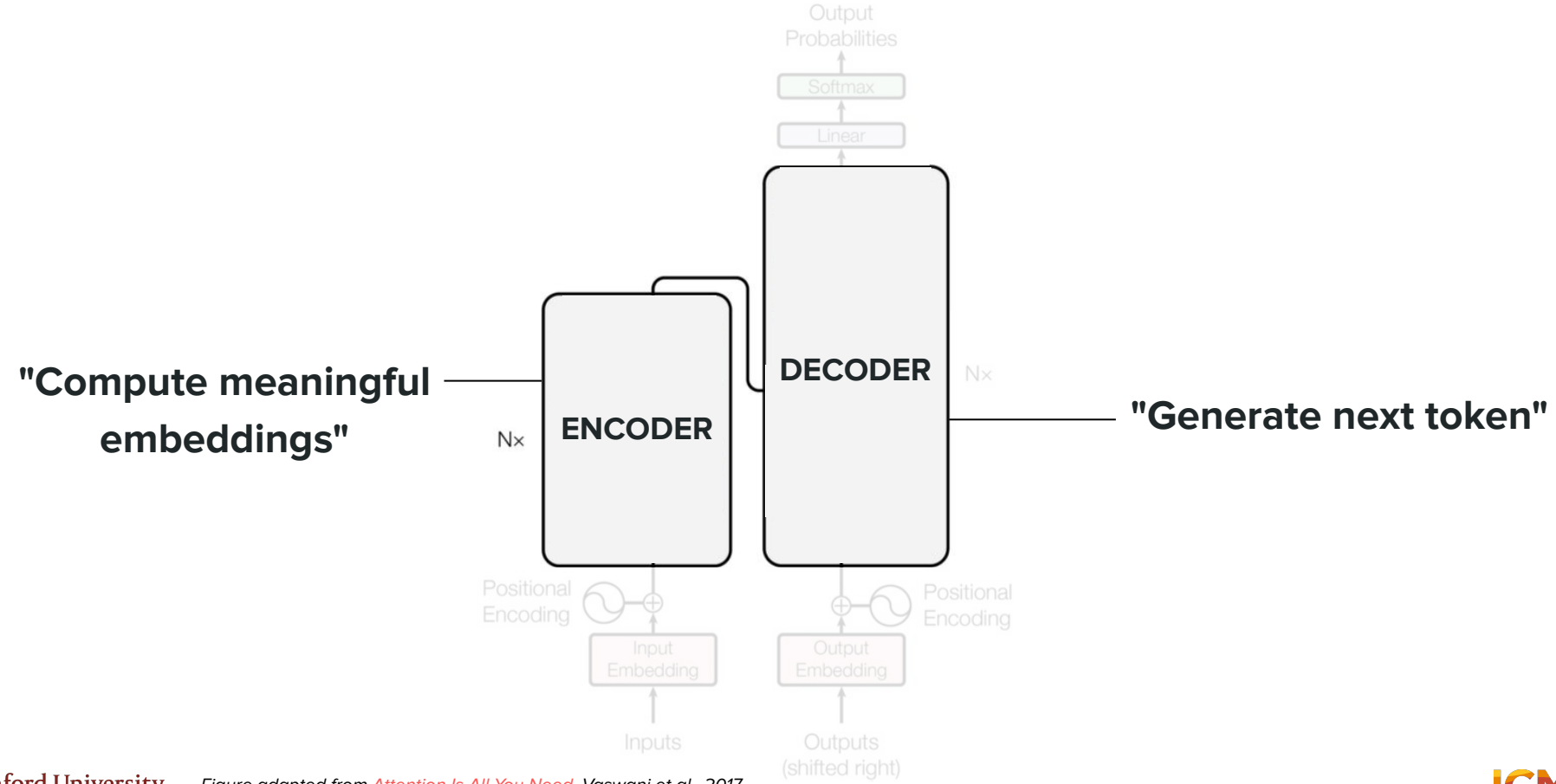


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Transformer variant: encoder-only

"Compute meaningful embeddings"

and use for downstream task

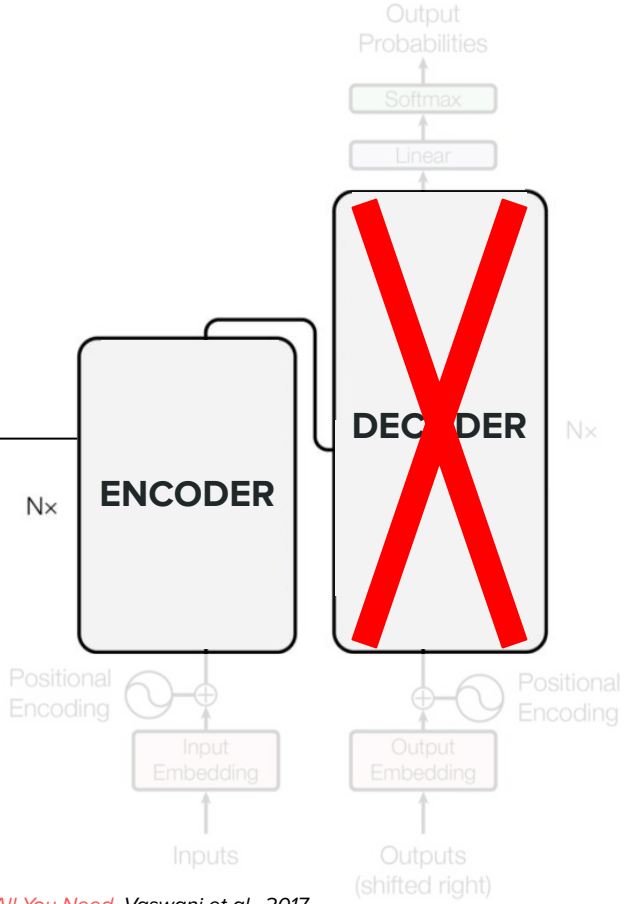
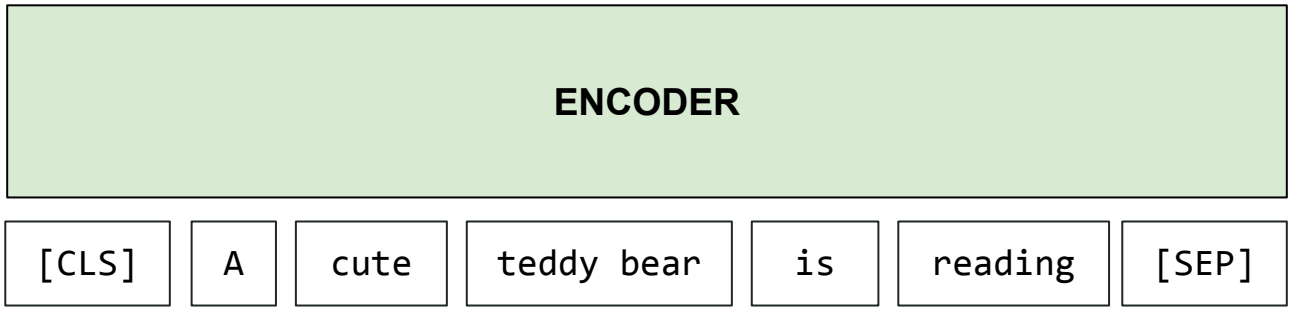


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

BERT and encoder-only models

BERT = **B**idirectional **E**ncoder **R**epresentations from **T**ransformers

Idea. Leverage **encoded embeddings** to make predictions.

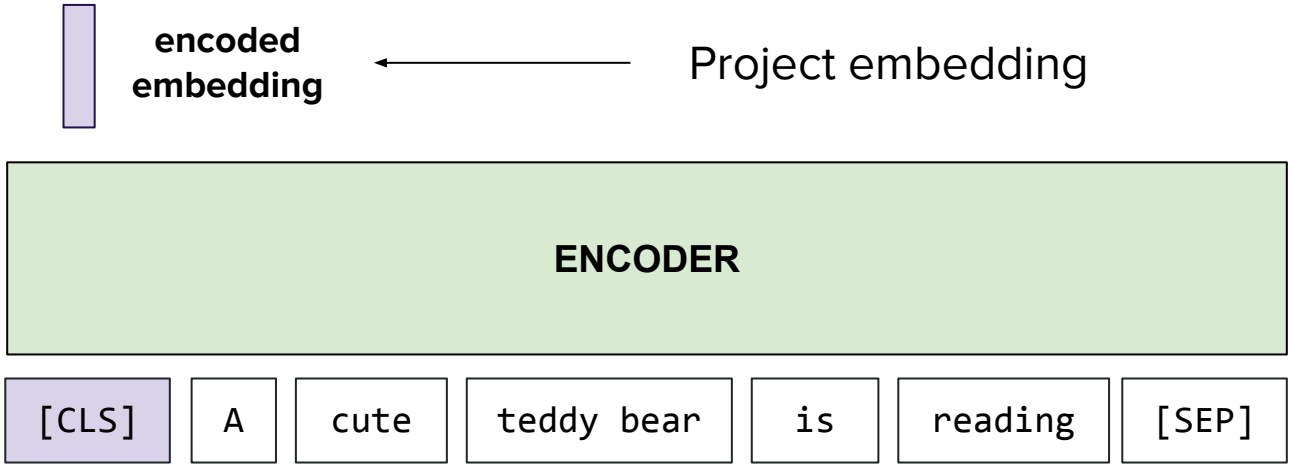


BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Devlin et al., 2018.

BERT and encoder-only models

BERT = **B**idirectional **E**ncoder **R**epresentations from **T**ransformers

Idea. Leverage **encoded embeddings** to make predictions.

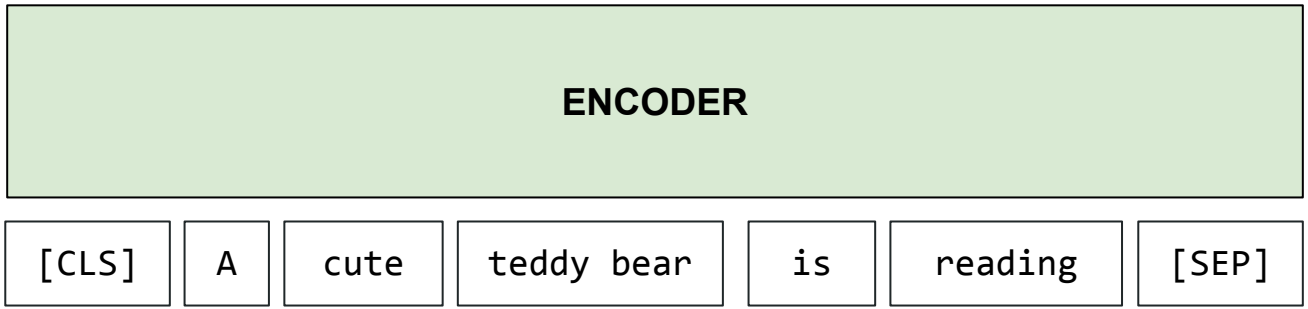


BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Devlin et al., 2018.

BERT and encoder-only models

Strategy.

- **Step 1**: Pretraining with proxy tasks: **M**asked **L**anguage **M**odel (MLM)

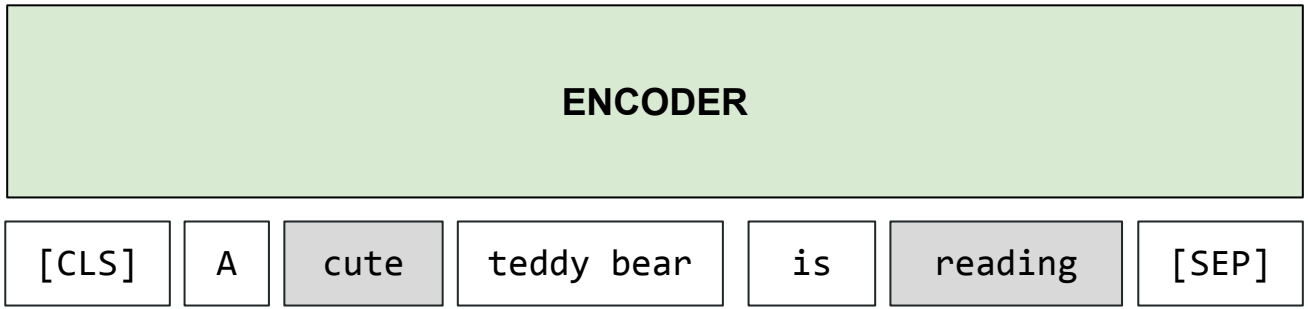


BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Devlin et al., 2018.

BERT and encoder-only models

Strategy.

- **Step 1**: Pretraining with proxy tasks: **M**asked **L**anguage **M**odel (MLM)

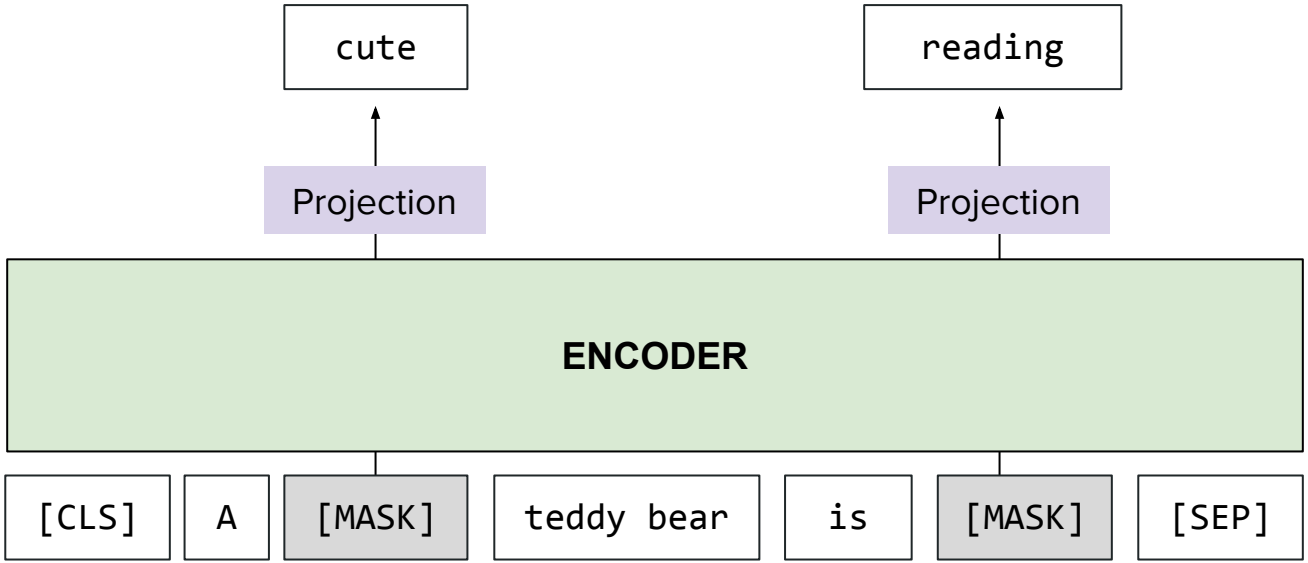


BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Devlin et al., 2018.

BERT and encoder-only models

Strategy.

- **Step 1**: Pretraining with proxy tasks: **M**asked **L**anguage **M**odel (MLM)

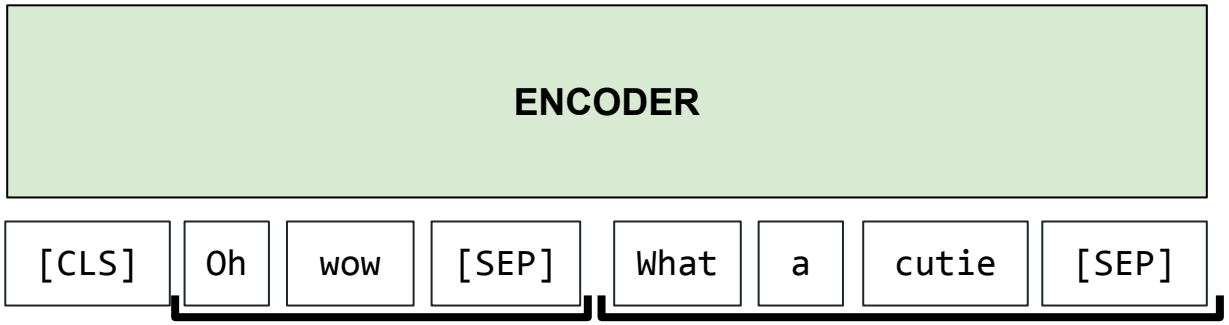


BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Devlin et al., 2018.

BERT and encoder-only models

Strategy.

- **Step 1**: Pretraining with proxy tasks: **N**ext **S**entence **P**rediction (NSP)

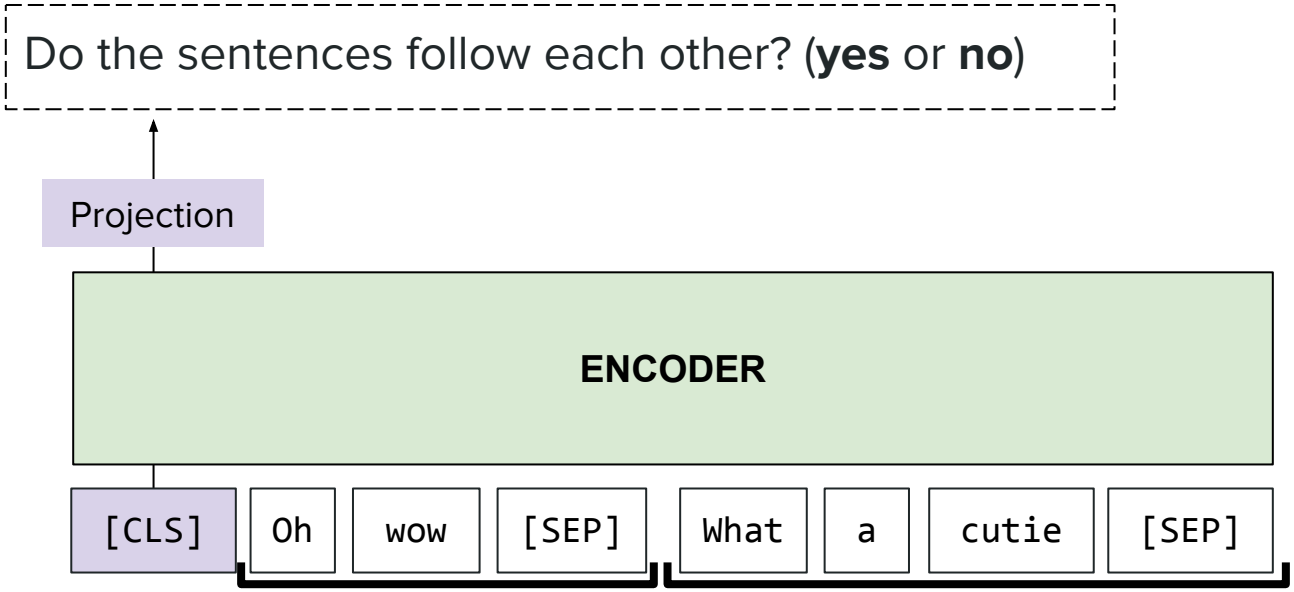


BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Devlin et al., 2018.

BERT and encoder-only models

Strategy.

- **Step 1**: Pretraining with proxy tasks: **N**ext **S**entence **P**rediction (NSP)

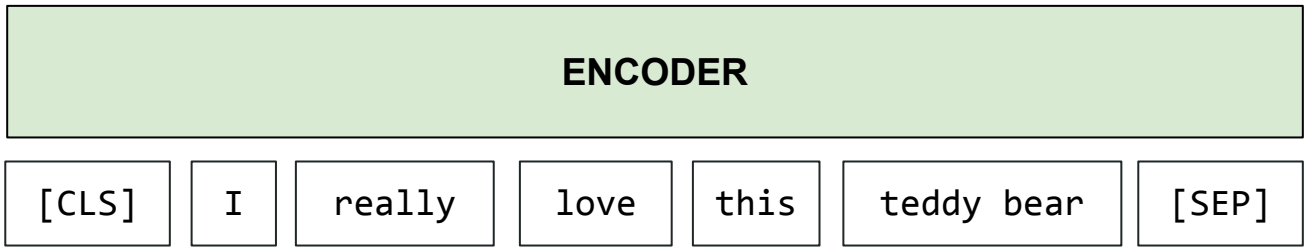


BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Devlin et al., 2018.

BERT and encoder-only models

Strategy.

- **Step 1**: Pretraining with proxy tasks (MLM and NSP)
- **Step 2**: Finetuning for given end task

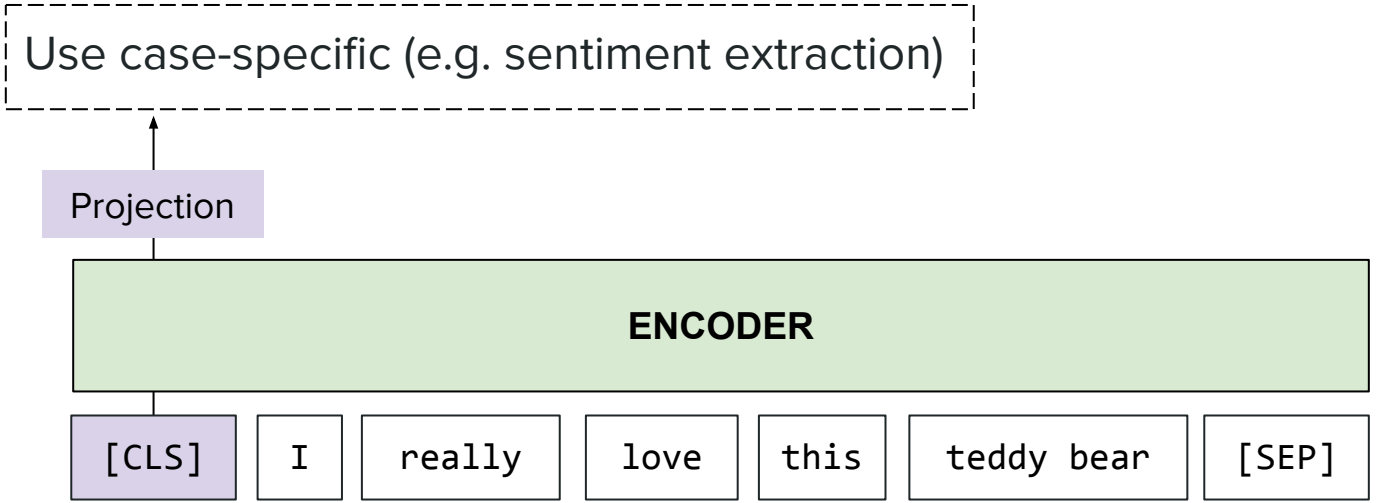


BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Devlin et al., 2018.

BERT and encoder-only models

Strategy.

- **Step 1**: Pretraining with proxy tasks (MLM and NSP)
- **Step 2**: Finetuning for given end task



BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Devlin et al., 2018.

BERT and encoder-only models

Strategy.

- **Step 1**: Pretraining with proxy tasks (MLM and NSP)
- **Step 2**: Finetuning for given end task

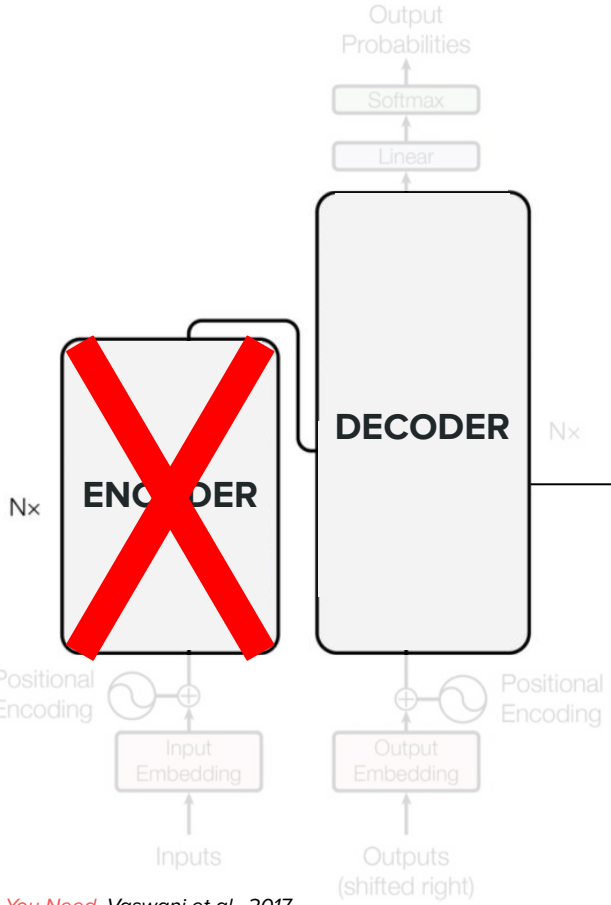
Discussion.

Pros	Cons
• Good performance • Finetuning does not need a lot of data	• Finetuning is a required step • Not suited for a range of tasks (e.g. text generation)

"Not suited for text generation" ⟶ until Lecture 8 🙂

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Devlin et al., 2018.

Transformer variant: decoder-only



"Generate next token"

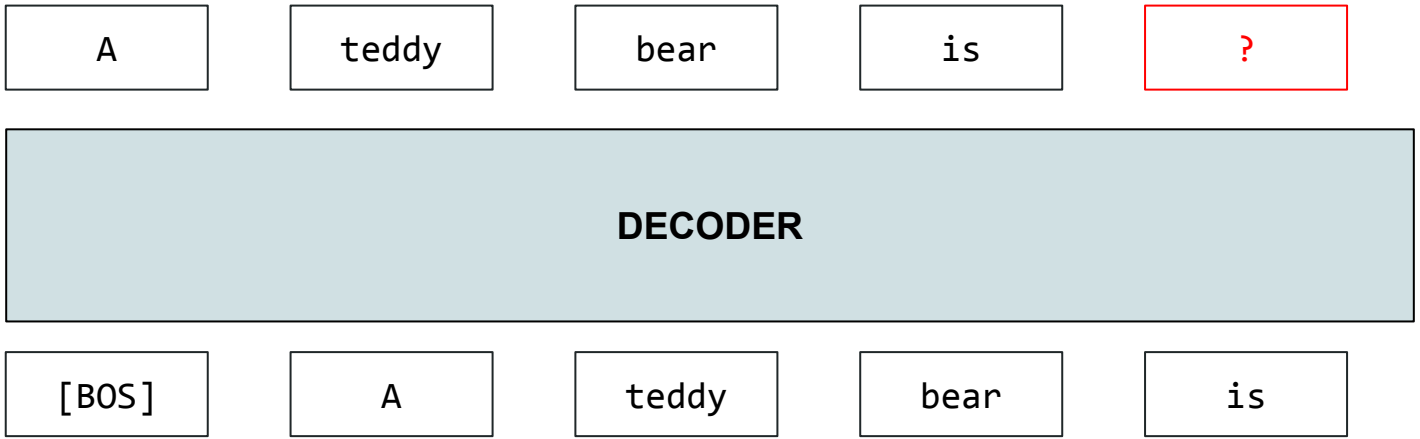
autoregressively

Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

GPT and decoder-only models

GPT = **G**enerative **P**re-trained **T**ransformer

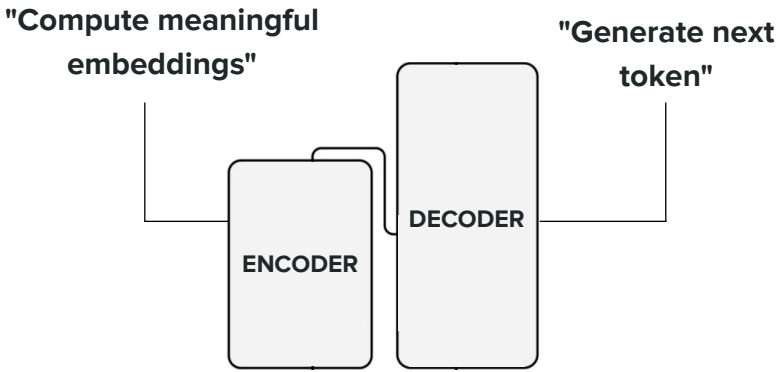
Idea. Predict next token one at a time.



Improving Language Understanding by Generative Pre-Training, Radford et al., 2018.

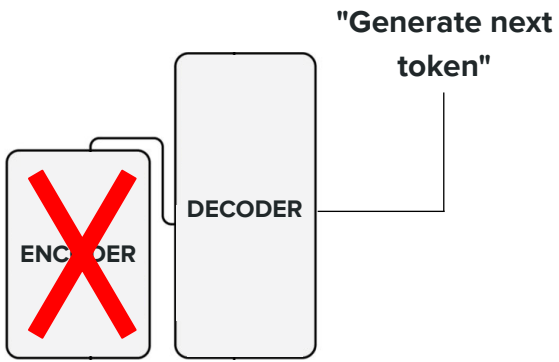
Encoder-decoder versus decoder-only

Encoder-decoder



- Input: **bidirectional** attention
- Output: causal attention

Decoder-only



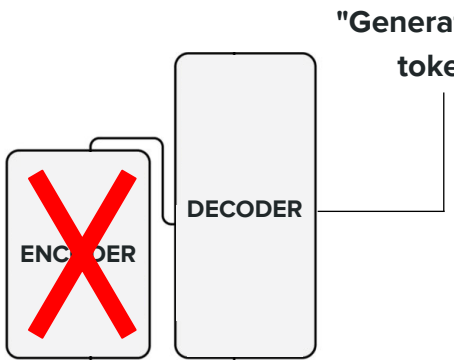
- Input: **causal** attention
- Output: causal attention

Encoder-decoder versus decoder-only

Encoder-decoder

- Input: **bidirectional** attention
- Output: causal attention

Decoder-only



- Input: **causal** attention
- Output: causal attention



Transformers & Large Language Models

Transformer model families

Scaling laws

Expert routing

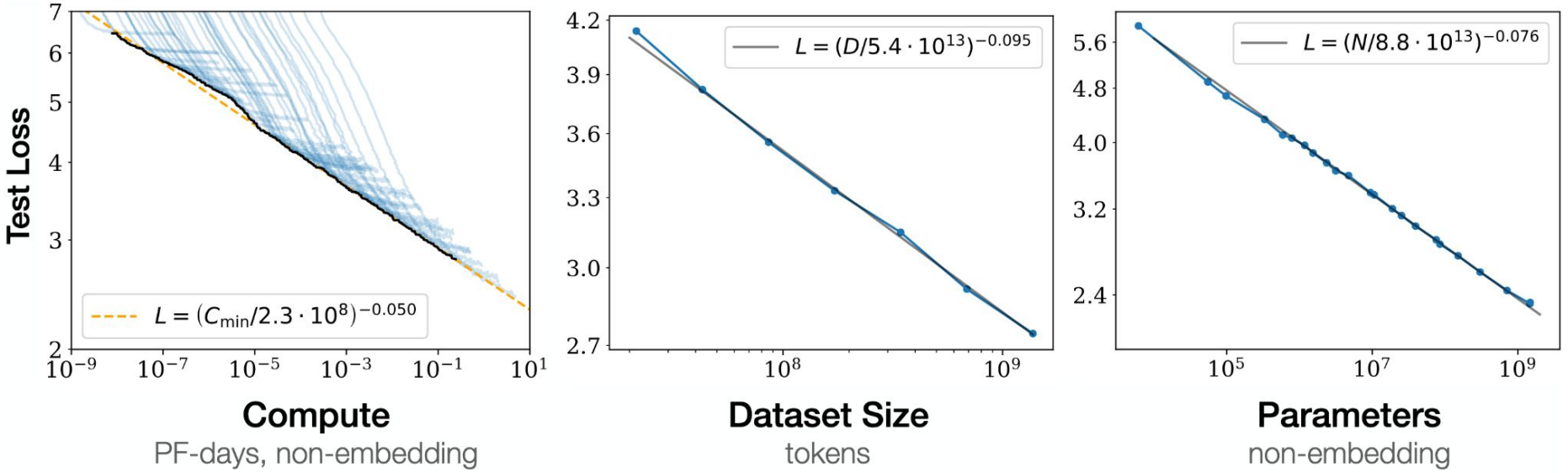
Architecture optimizations

Decoding strategies

Prompting and reasoning

Better performance with bigger models and more data

Scaling.

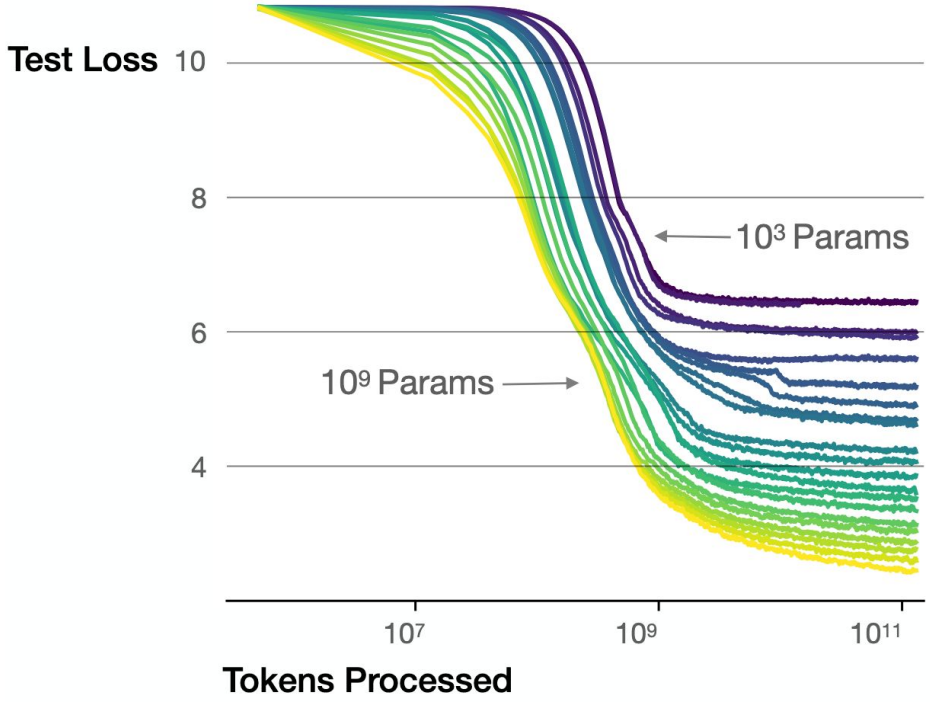


Scaling Laws for Neural Language Models, Kaplan et al., 2020.

Bigger models are more token efficient

Scaling.

Sample efficiency.



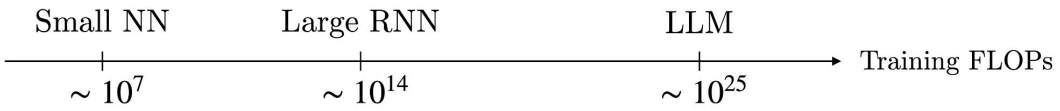
Scaling Laws for Neural Language Models, Kaplan et al., 2020.

Notations

📖 **FLOPs**

FL**oating-point **OP**erations**s

$f(\#\text{tokens}, \#\text{parameters})$



Notations

📖 **FLOPs**

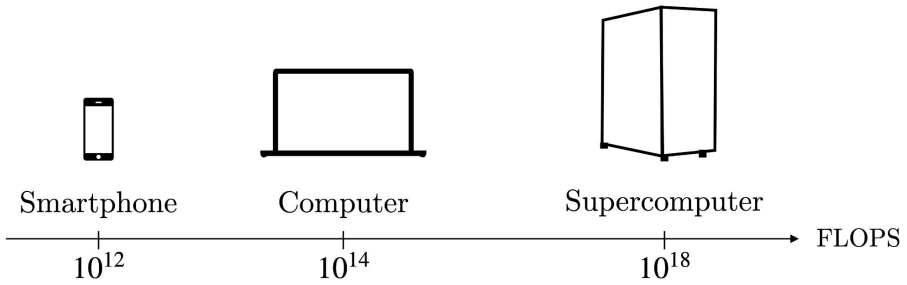
***FL**oating-point **OP**erations*

$f(\#\text{tokens}, \#\text{parameters})$

⚡ **FLOPS** or **FLOP/s**

***FL**oating-point **OP**erations per **S**econd*

Computational performance

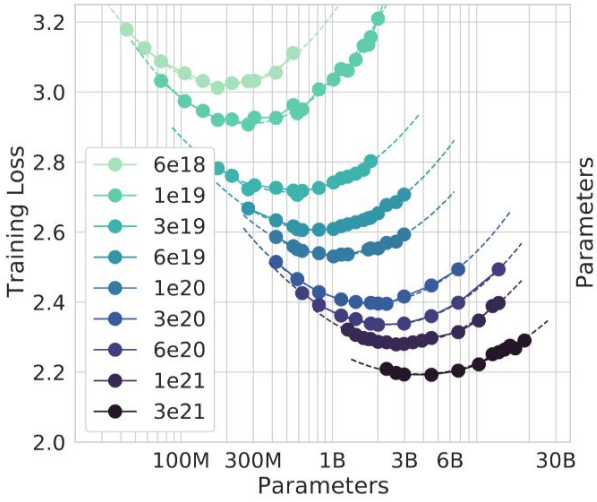


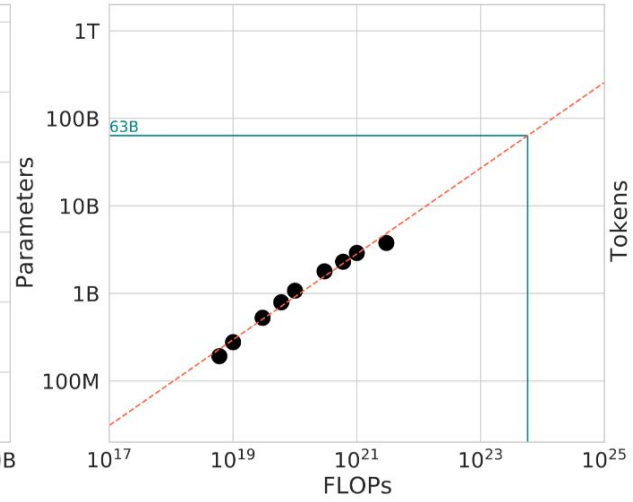
Takeaways on compute-optimal training

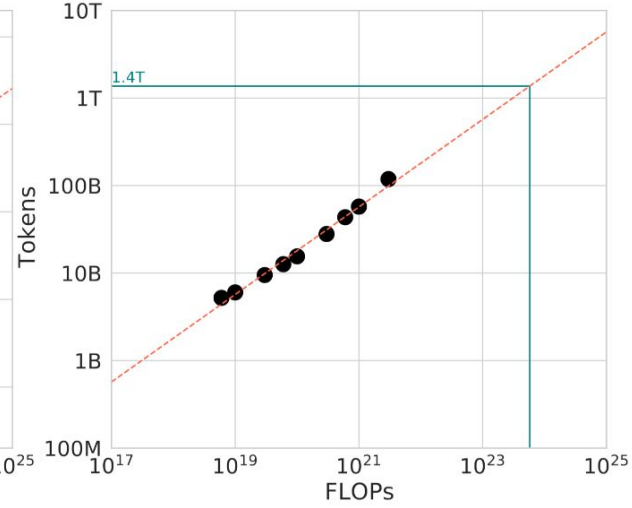
Scaling.

Sample efficiency.

Chinchilla law.







Training Compute-Optimal Large Language Models, Hoffmann et al., 2022.

Takeaways on compute-optimal training

Scaling.

Sample efficiency.

Chinchilla law.

x 20

Parameters	FLOPs	FLOPs (in *Gopher* unit)	Tokens
400 Million	1.92e+19	1/29,968	8.0 Billion
1 Billion	1.21e+20	1/4,761	20.2 Billion
10 Billion	1.23e+22	1/46	205.1 Billion
67 Billion	5.76e+23	1	1.5 Trillion
175 Billion	3.85e+24	6.7	3.7 Trillion
280 Billion	9.90e+24	17.2	5.9 Trillion
520 Billion	3.43e+25	59.5	11.0 Trillion
1 Trillion	1.27e+26	221.3	21.2 Trillion
10 Trillion	1.30e+28	22515.9	216.2 Trillion

Training Compute-Optimal Large Language Models, Hoffmann et al., 2022.

Terminology

LLM = **L**arge **L**anguage **M**odel

Terminology

LLM = Large **L**anguage **M**odel

"A **language model** is a statistical or machine learning **model** that assigns **probabilities** to sequences of **tokens**."

Terminology

LLM = Large Language Model

- **Model size**: billions of parameters or more
- **Training data**: 100s of billions of tokens or more
- **Compute**: a lot of GPUs

Characteristics

Decoder-only **Transformer**-based model

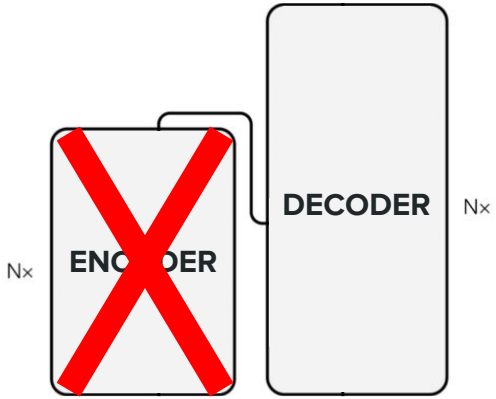


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Characteristics

Decoder-only **Transformer**-based model

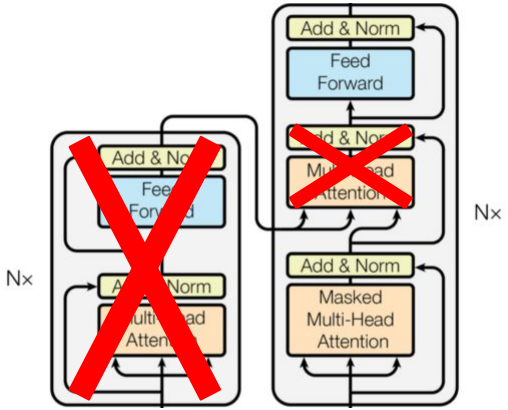
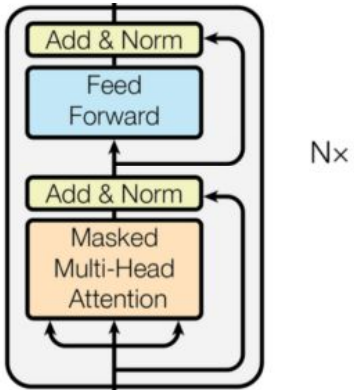


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Characteristics

Decoder-only **Transformer**-based model



Examples: GPT series, Gemma, DeepSeek, Qwen, GLM, Kimi, ...

Figure adapted from Attention Is All You Need, Vaswani et al., 2017.



Transformers & Large Language Models

Transformer model families

Scaling laws

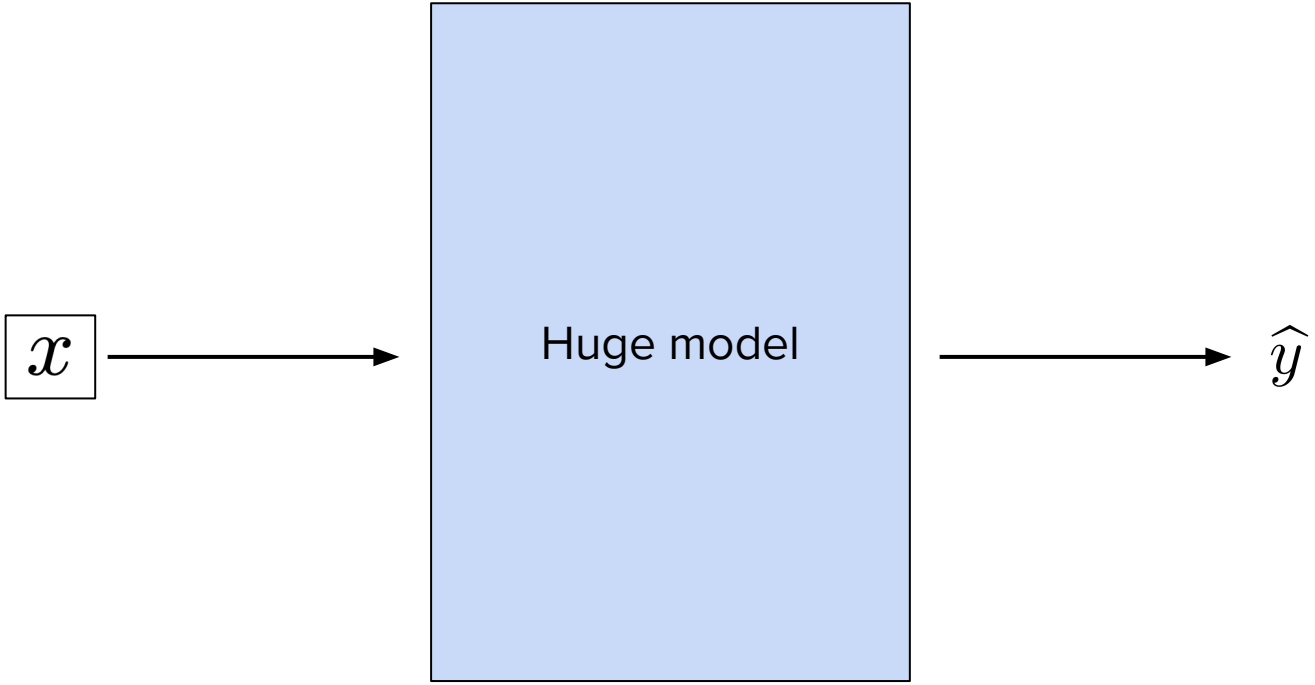
Expert routing

Architecture optimizations

Decoding strategies

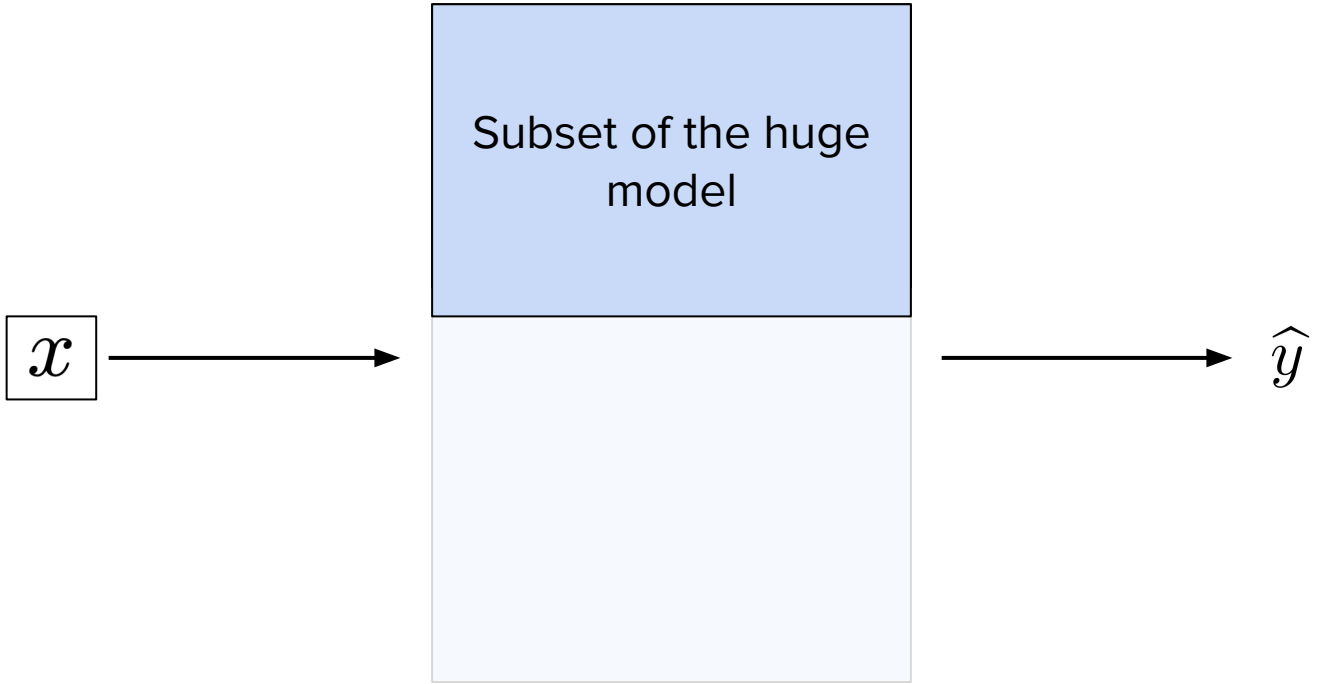
Prompting and reasoning

Motivation

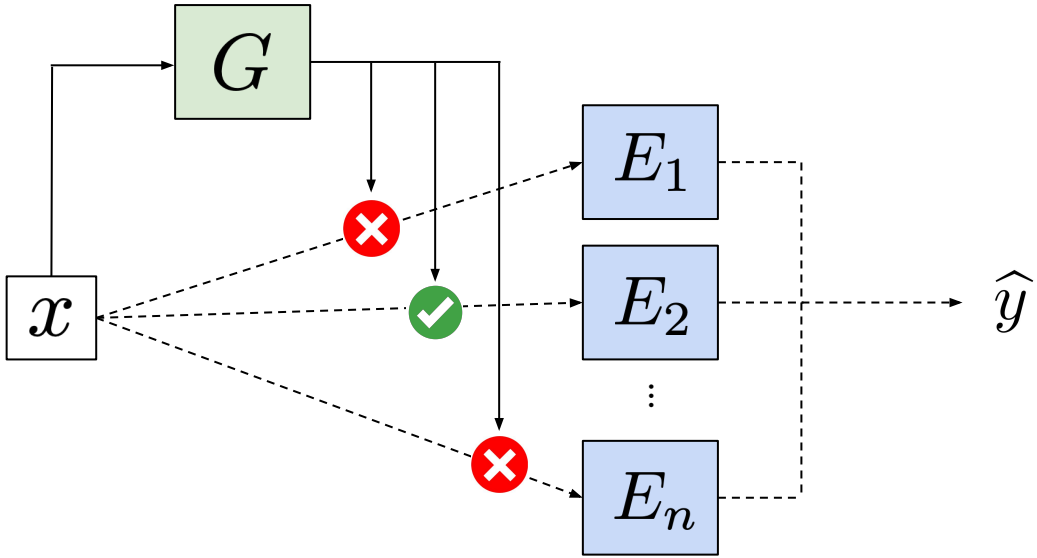


Motivation

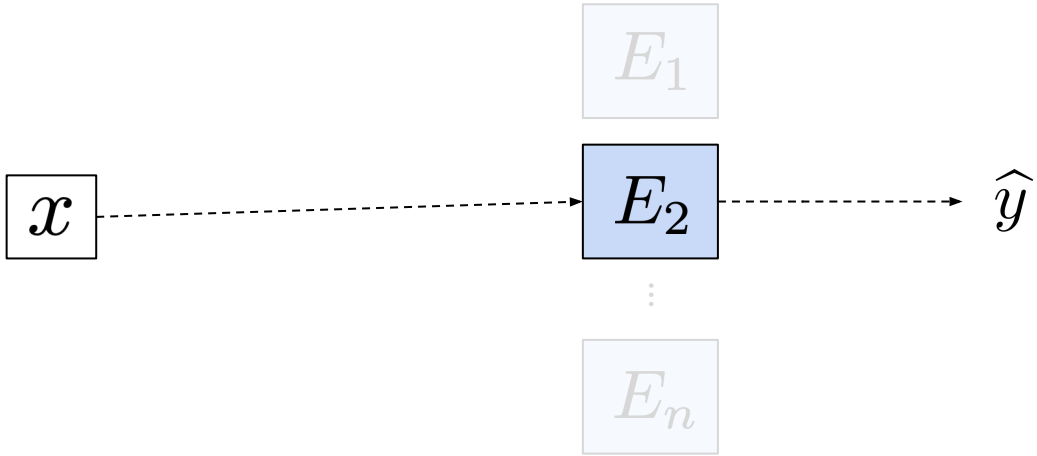
Idea. Not all weights are useful in the forward pass



Motivation

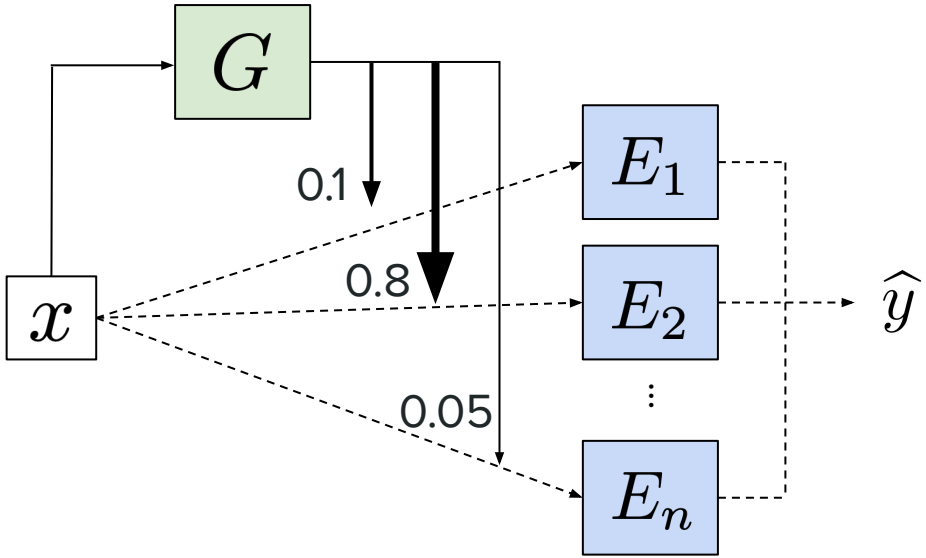


Motivation



Overview of MoEs

MoE = **M**ixture **o**f **E**xperts

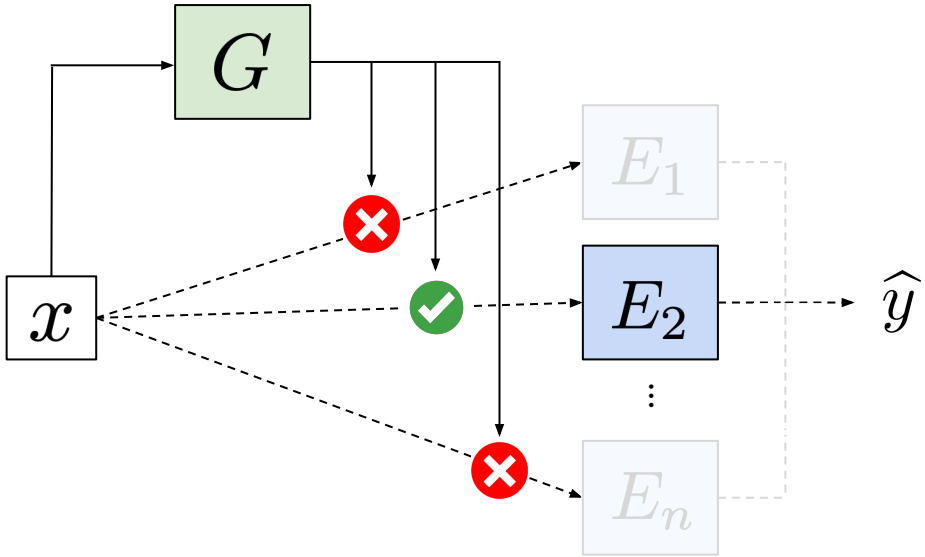


Dense MoE. Output is weighted average of **all** expert outputs.

$$\widehat{y} = \sum_{i=1}^{n} G(x)_i E_i(x)$$

Overview of MoEs

MoE = **M**ixture **o**f **E**xperts



Sparse MoE. Output is weighted average of **selected** expert outputs.

$$\widehat{y} = \sum_{i \in \mathcal{I}_k} G(x)_i E_i(x)$$

Via **top-k** selection

Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer, Shazeer et al., 2017.

MoE in Transformer-based models

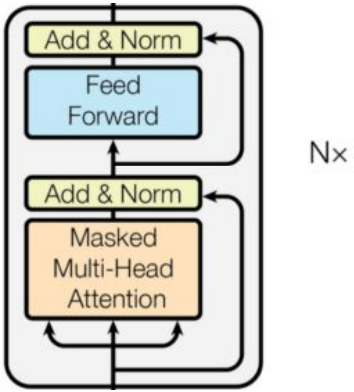
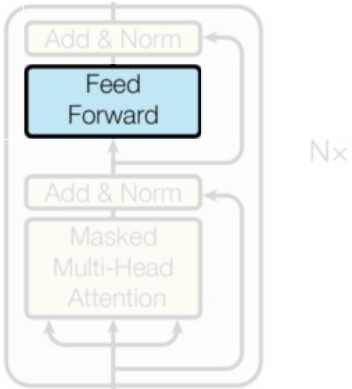


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

MoE in Transformer-based models



Routing done **for each token**!

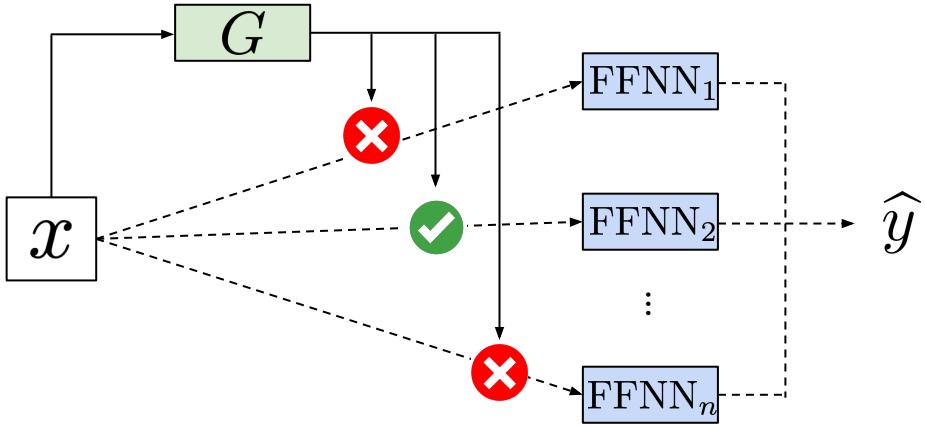


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Training challenges include routing collapse

Symptom. Same expert gets selected most of the time.

"routing collapse"

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity, Fedus et al., 2021.

Training challenges include routing collapse

Symptom. Same expert gets selected most of the time.

"routing collapse"

Remedy. Force other experts to be "part of the game" too via auxiliary loss:

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i$$

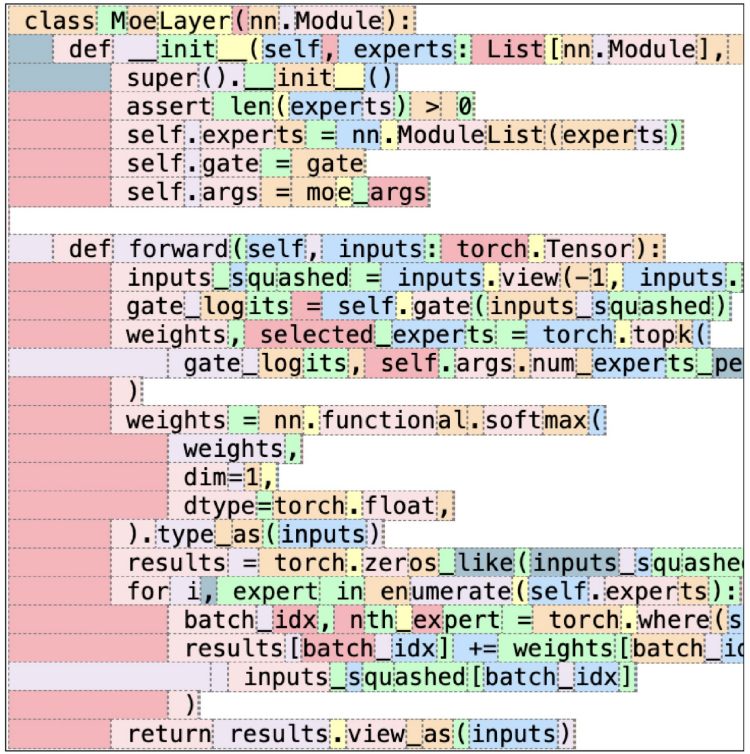
f_i = fraction of tokens routed to expert i

P_i = average routing probability for expert i

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity, Fedus et al., 2021.

Interpreting experts

Layer 0



```
class MoeLayer(nn.Module):
    def __init__(self, experts: List[nn.Module],
        super().__init__()
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = moe_args

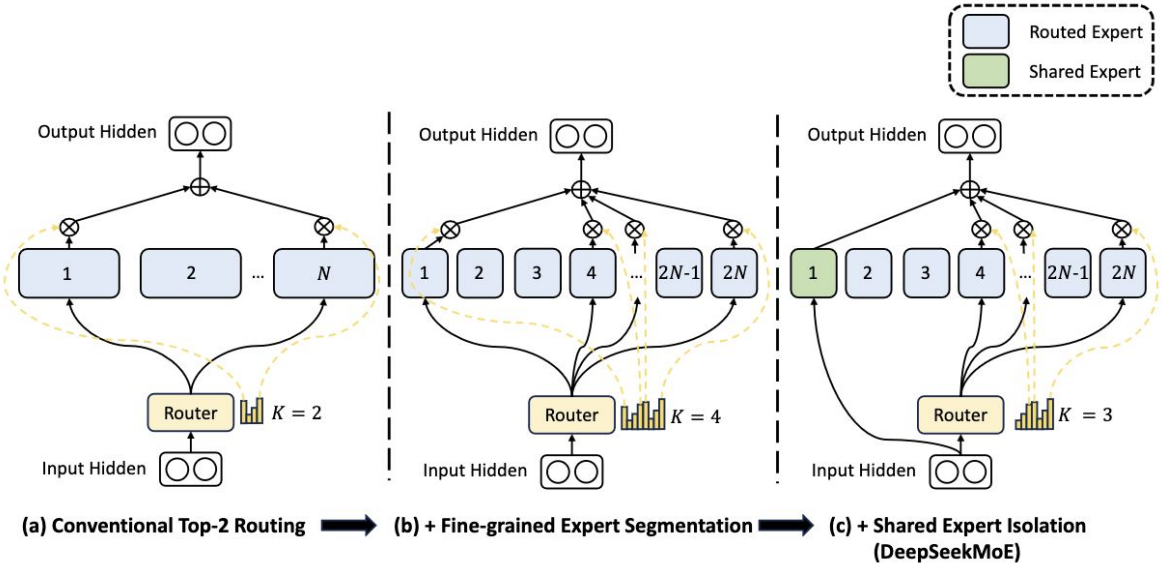
    def forward(self, inputs: torch.Tensor):
        inputs_squashed = inputs.view(-1, inputs.
        gate_logits = self.gate(inputs_squashed)
        weights, selected_experts = torch.topk(
            gate_logits, self.args.num_experts_pe
        )
        weights = nn.functional.softmax(
            weights,
            dim=1,
            dtype=torch.float,
        ).type_as(inputs)
        results = torch.zeros_like(inputs_squashe
        for i, expert in enumerate(self.experts):
            batch_idx, nth_expert = torch.where(s
            results[batch_idx] += weights[batch_id
                inputs_squashed[batch_idx]
            )
        return results.view_as(inputs)
```

Each color represents an expert

Mixtral of Experts, Jiang et al., 2024.

Recent trends

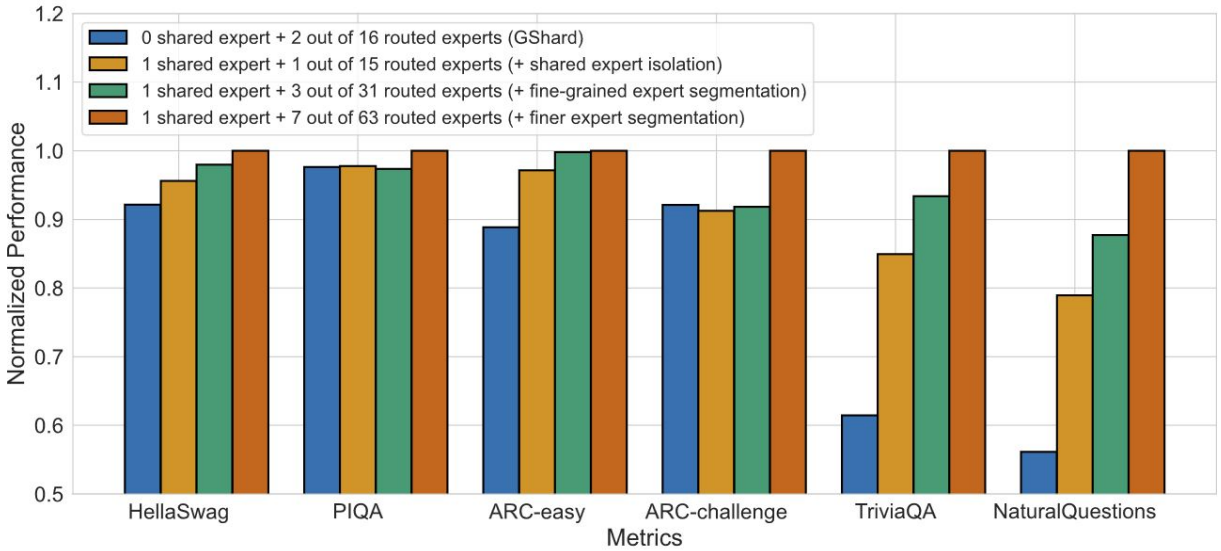
- **Shared** experts to **minimize redundant information** between experts



DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models, Dai et al., 2024.

Recent trends

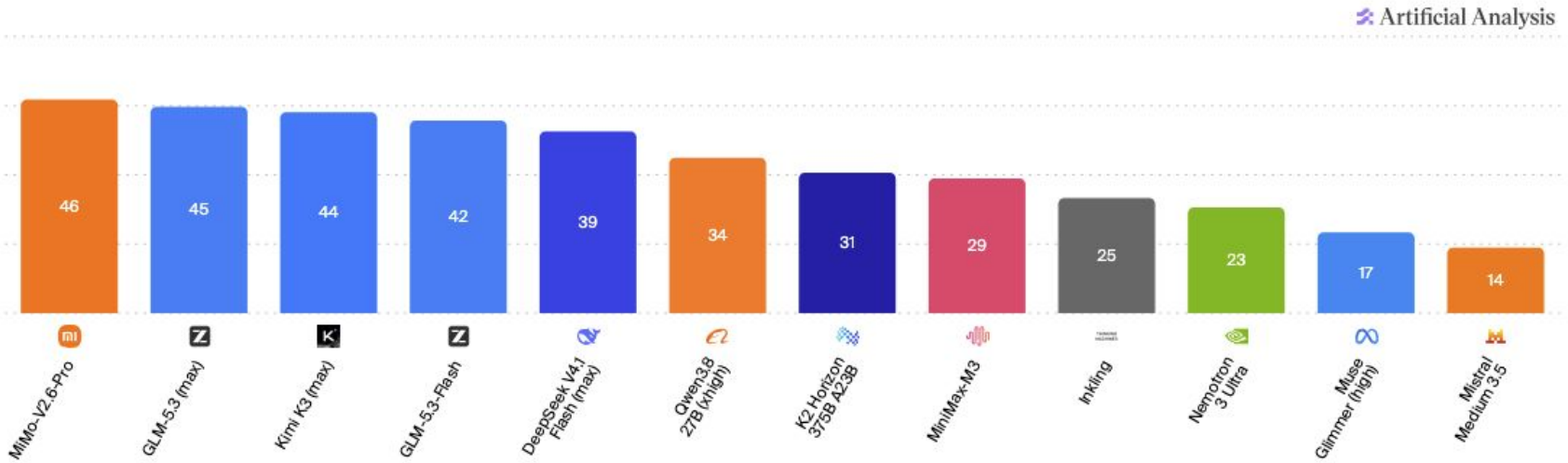
- **Shared** experts to **minimize redundant information** between experts
- **Fine-grained** experts with top k **> 1**



DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models, Dai et al., 2024.

Recent trends

- **Shared** experts to **minimize redundant information** between experts
- **Fine-grained** experts with top k **> 1**
- Orders of magnitude among **frontier open-source models**



Screenshot from Artificial Analysis taken on September 27th, 2026.

Recent trends

- **Shared** experts to **minimize redundant information** between experts
- **Fine-grained** experts with top k **> 1**
- Orders of magnitude among **frontier open-source models**

	MiMo-V2.6 Pro	GLM-5	DeepSeek V4.1 Flash
# **total** experts	384	256	384
# **routed** experts	8	8	6
# **shared** experts	N/A	1	1
Model **size**	1.02T	744B	552B

MiMo-v2.6: Scaling Reinforcement Learning Towards Self-Improvement, *LLM-Core Xiaomi, September 2026.*
GLM-5: from Vibe Coding to Agentic Engineering, *GLM-5 Team, February 2026.*
DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression, *DeepSeek AI, September 2026.*



Transformers & Large Language Models

Transformer model families

Scaling laws

Expert routing

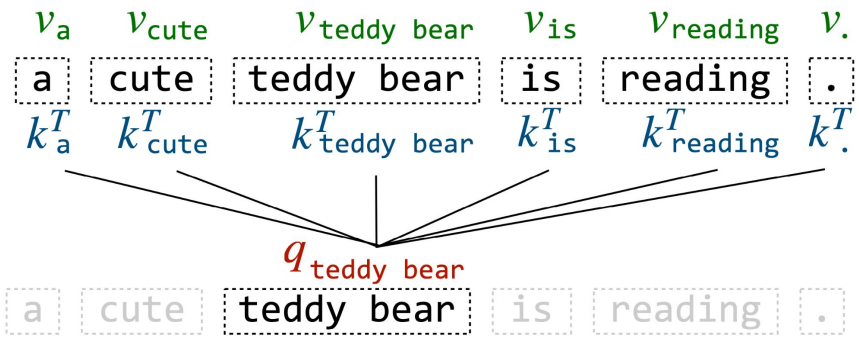
Architecture optimizations

Decoding strategies

Prompting and reasoning

Need for position information

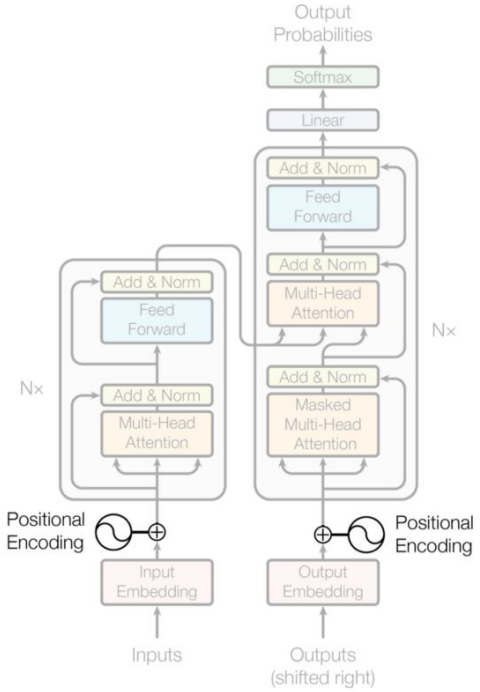
Motivation. Direct links "lose" position info



Need for position information

Motivation. Direct links "lose" position info

Idea. Add learned position-specific embedding to token vector



Attention Is All You Need, Vaswani et al., 2017.

Need for position information

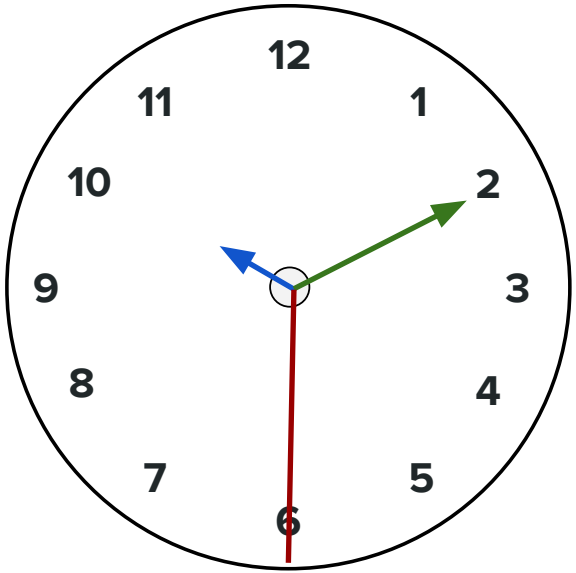
Motivation. Direct links "lose" position info

Idea. Add learned position-specific embedding to token vector

Limitations. Need to retrain for longer sequences

Hardcoded position embeddings

Idea. Hardcode values of position embedding



Hour (slow)

Minute (fast)

Second (very fast)

Hardcoded position embeddings

Idea. Hardcode values of position embedding

Hour (slow)

Minute (fast)

Second (very fast)

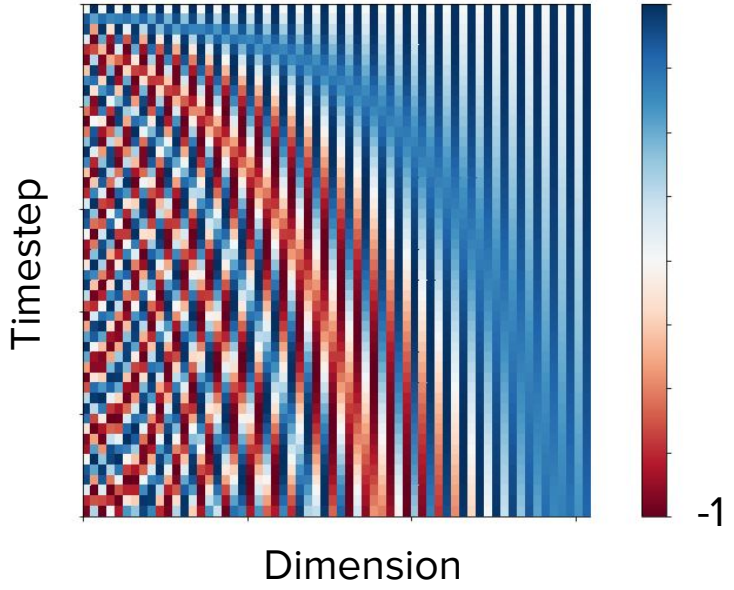
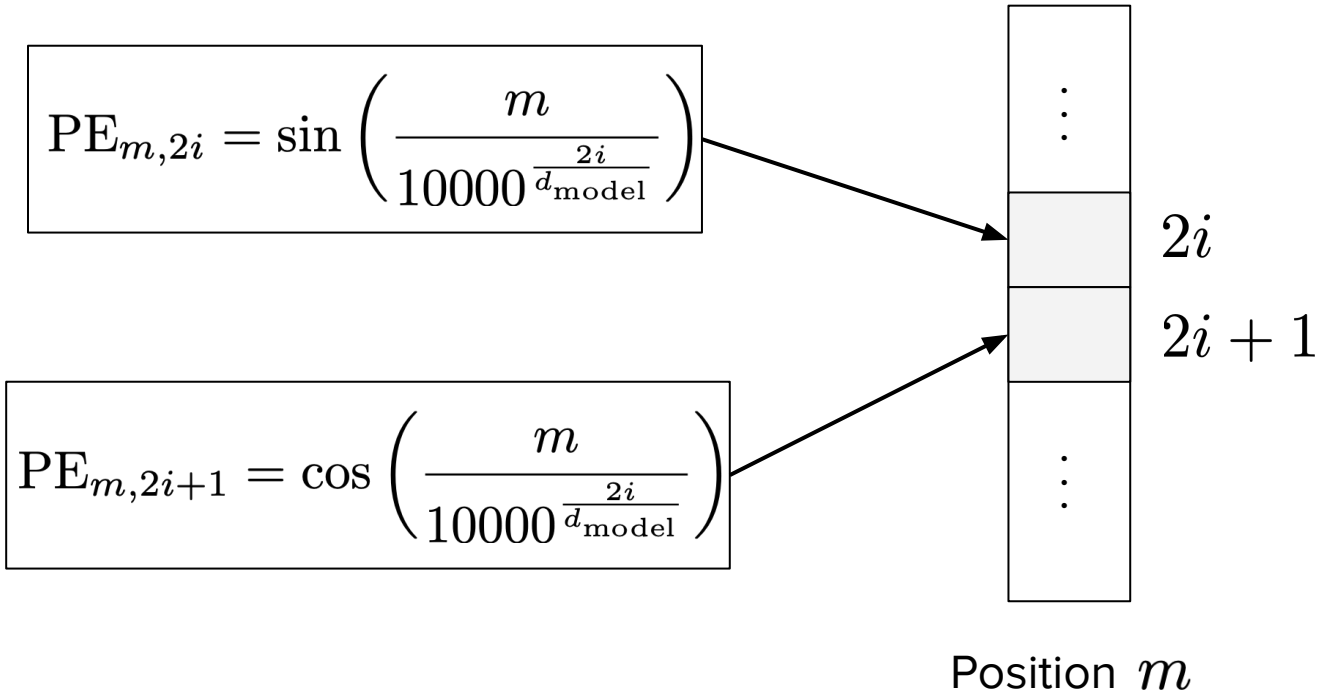


Figure adapted from Transformer Architecture: The Positional Encoding, Kazemnejad, 2019.

Hardcoded position embeddings

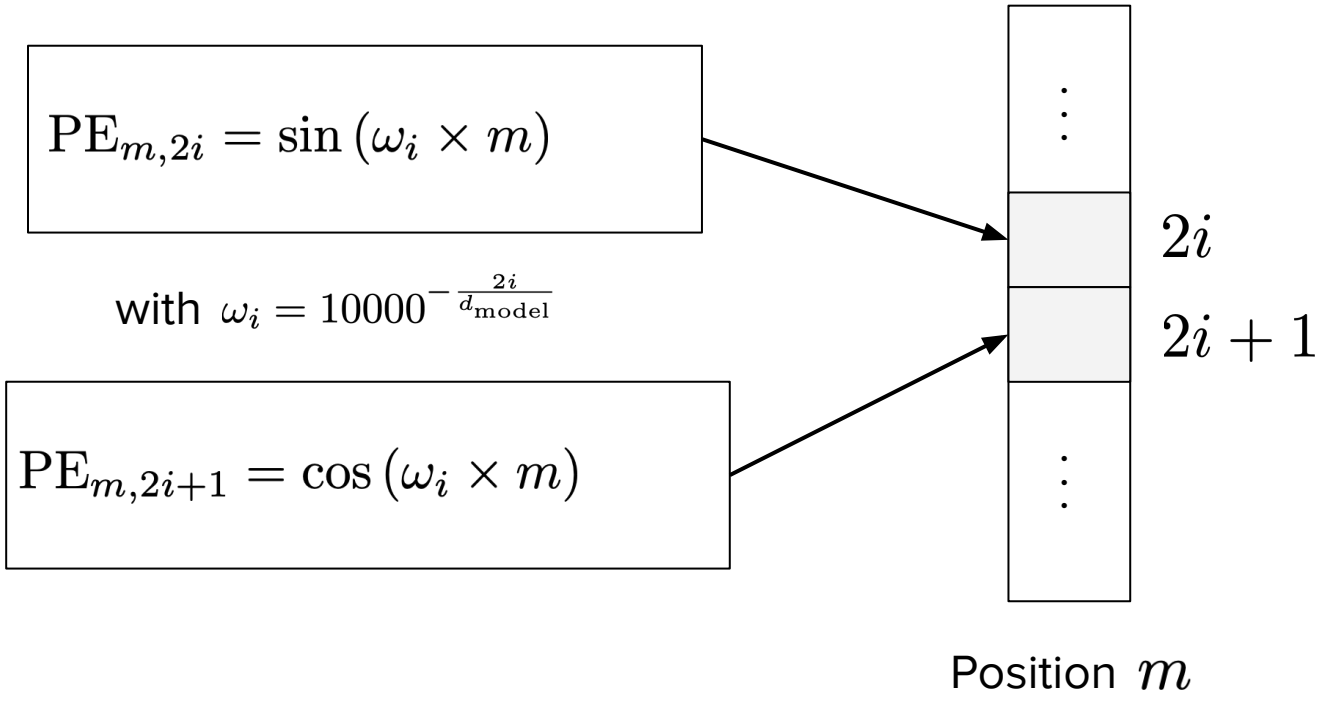
Idea. Hardcode values of position embedding



Attention Is All You Need, Vaswani et al., 2017.

Hardcoded position embeddings

Idea. Hardcode values of position embedding

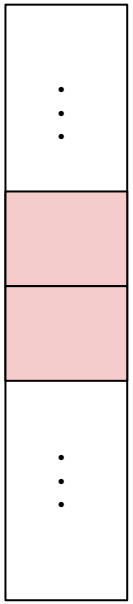


Attention Is All You Need, Vaswani et al., 2017.

Hardcoded position embeddings

Idea. Hardcode values of position embedding

$$\text{PE}_{m,2i} = \sin\left(\omega_i \times m\right)$$
$$\text{PE}_{m,2i+1} = \cos\left(\omega_i \times m\right)$$

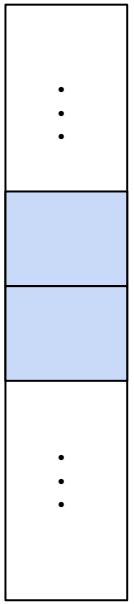


$2i$

$2i+1$

Position m

$$\text{PE}_{n,2i} = \sin\left(\omega_i \times n\right)$$
$$\text{PE}_{n,2i+1} = \cos\left(\omega_i \times n\right)$$



$2i$

$2i+1$

Position n

Attention Is All You Need, Vaswani et al., 2017, *Transformer Architecture: The Positional Encoding*, Kazemnejad, 2019.

Hardcoded position embeddings

Idea. Hardcode values of position embedding

$$\cos(a-b) = \cos(a)\cos(b) + \sin(a)\sin(b)$$

Hardcoded position embeddings

Idea. Hardcode values of position embedding

$$\cos(a-b) = \cos(a)\cos(b) + \sin(a)\sin(b)$$

$$\cos(\omega_i(m-n)) = \cos(\omega_i \times m)\cos(\omega_i \times n) + \sin(\omega_i \times m)\sin(\omega_i \times n)$$

Hardcoded position embeddings

Idea. Hardcode values of position embedding

$$\cos(a-b) = \cos(a)\cos(b) + \sin(a)\sin(b)$$

$$\cos(\omega_i(m-n)) = \cos(\omega_i \times m)\cos(\omega_i \times n) + \sin(\omega_i \times m)\sin(\omega_i \times n)$$

$$\langle \mathrm{PE}_m, \mathrm{PE}_n \rangle = \ldots + \cos(\omega_i(m-n)) + \ldots$$

Hardcoded position embeddings

Idea. Hardcode values of position embedding

$$\cos(a-b) = \cos(a)\cos(b) + \sin(a)\sin(b)$$

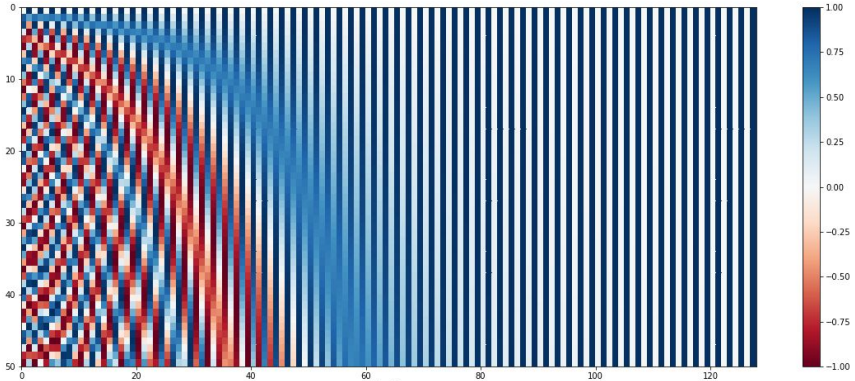
$$\cos(\omega_i(m-n)) = \cos(\omega_i \times m)\cos(\omega_i \times n) + \sin(\omega_i \times m)\sin(\omega_i \times n)$$

$$\langle \mathrm{PE}_m, \mathrm{PE}_n \rangle = \ldots + \cos(\omega_i(m-n)) + \ldots$$

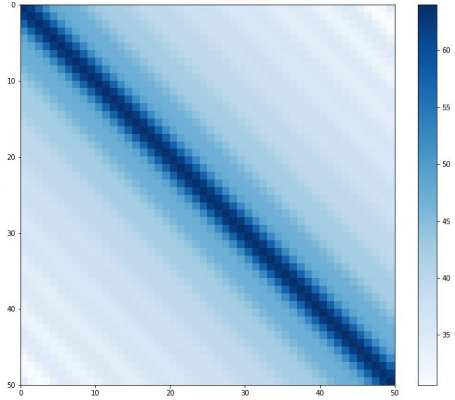
$$\langle \mathrm{PE}_m, \mathrm{PE}_n \rangle = f(m-n)$$

Hardcoded position embeddings

Idea. Hardcode values of position embedding



Value of embeddings



Similarity between positions

Attention Is All You Need, Vaswani et al., 2017, *Transformer Architecture: The Positional Encoding*, Kazemnejad, 2019.

Hardcoded position embeddings

Idea. Hardcode values of position embedding

Value of embeddings

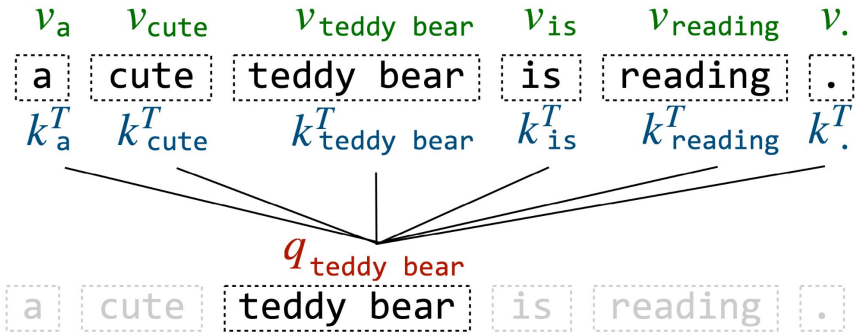
Similarity between positions

Benefits. Can extend to any sequence length

Attention Is All You Need, Vaswani et al., 2017, *Transformer Architecture: The Positional Encoding*, Kazemnejad, 2019.

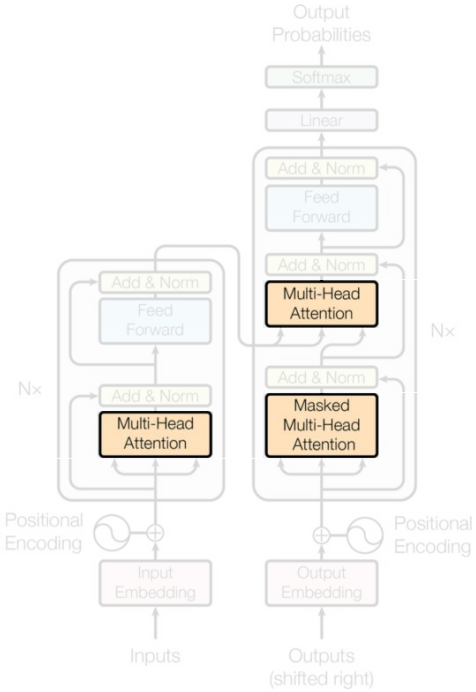
From absolute to relative position info

Motivation. Actually, we really care about relative position **with attention layer**



From absolute to relative position info

Motivation. Actually, we really care about relative position **with attention layer**

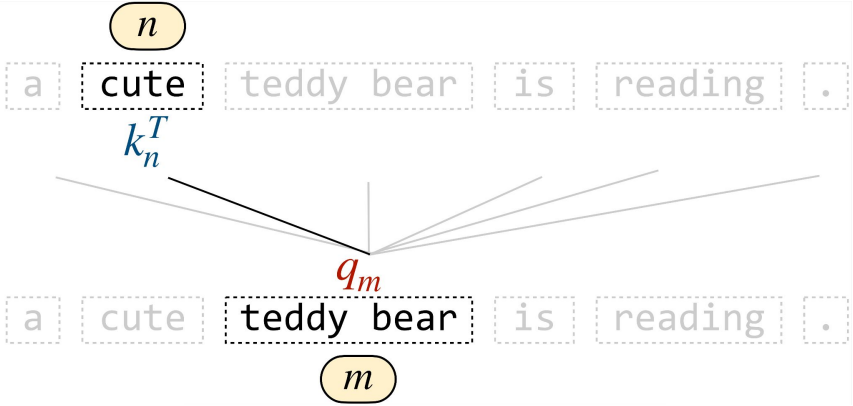


Idea. Let's change the attention layer instead

Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Linear bias in attention layer

Idea. Add bias to query-key scores



$$\text{softmax}\left(\frac{\langle q_m, k_n \rangle}{\sqrt{d_k}} + \text{bias}(m, n)\right)$$

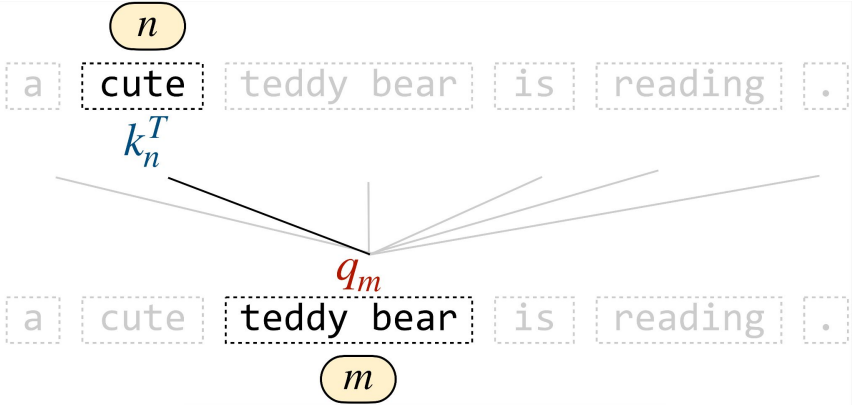
T5 bias. Bias is **learned per head**

$$\text{bias}(m, n) = \beta_{\text{bucket}(m-n)}$$

Super Study Guide: Transformers & Large Language Models, Amidi, 2024.
Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer, Raffel et al., 2023.

Linear bias in attention layer

Idea. Add bias to query-key scores



$$\text{softmax}\left(\frac{\langle q_m, k_n \rangle}{\sqrt{d_k}} + \text{bias}(m,n)\right)$$

T5 bias. Bias is **learned per head**

$$\text{bias}(m,n) = \beta_{\text{bucket}(m-n)}$$

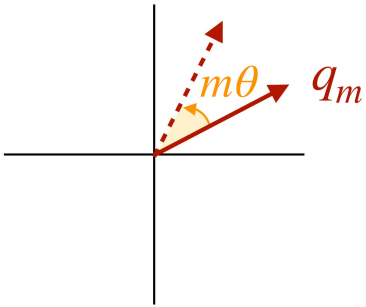
ALiBi. Bias is **linear**, **deterministic**, not bounded

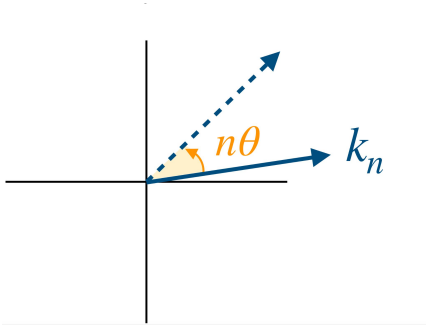
$$\text{bias}(m,n) = \mu \times (n-m)$$

Super Study Guide: Transformers & Large Language Models, Amidi, 2024.
Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation, Press et al., 2021.

Default choice nowadays: rotations in attention layer

Idea. Rotate query and key vectors with rotation matrix



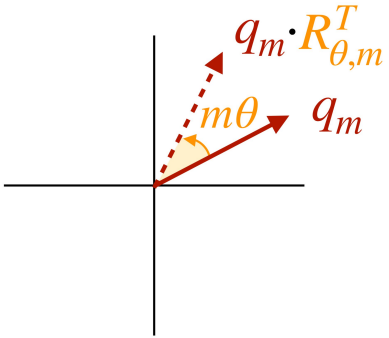


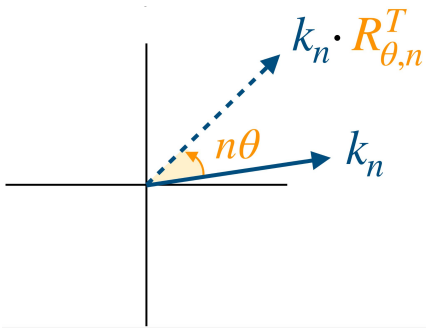
RoFormer: Enhanced Transformer with Rotary Position Embeddings, Su et al., 2021.
Figure from "Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Default choice nowadays: rotations in attention layer

RoPE = **Ro**tary **P**osition **E**mbeddings

Idea. Rotate query and key vectors with rotation matrix





Formula. Use rotation matrix: $R_{\theta,m} = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix}$

RoFormer: Enhanced Transformer with Rotary Position Embeddings, Su et al., 2021.
Figure from "Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Default choice nowadays: rotations in attention layer

Extension to dimension d > 2. Rotate every block of dimension 2

$$R_{\theta,m} = \begin{pmatrix} \text{block}_1 & 0 & \cdots & 0 \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & \text{block}_{\frac{d_k}{2}} \end{pmatrix} \qquad \text{block}_i = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}$$

RoFormer: Enhanced Transformer with Rotary Position Embeddings, Su et al., 2021.

Default choice nowadays: rotations in attention layer

Extension to dimension d > 2. Rotate every block of dimension 2

$$R_{\theta,m} = \begin{pmatrix} \text{block}_1 & 0 & \cdots & 0 \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & \text{block}_{\frac{d_k}{2}} \end{pmatrix} \qquad \text{block}_i = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}$$

Benefits. Relative distance nicely captured:

$$q_m k_n^T = x_m W_q R_{\theta,n-m} W_k^T x_n^T$$

RoFormer: Enhanced Transformer with Rotary Position Embeddings, Su et al., 2021.

Attention weight has a long-term decay

Relative upper bound of $A(q_m k_n^T)$

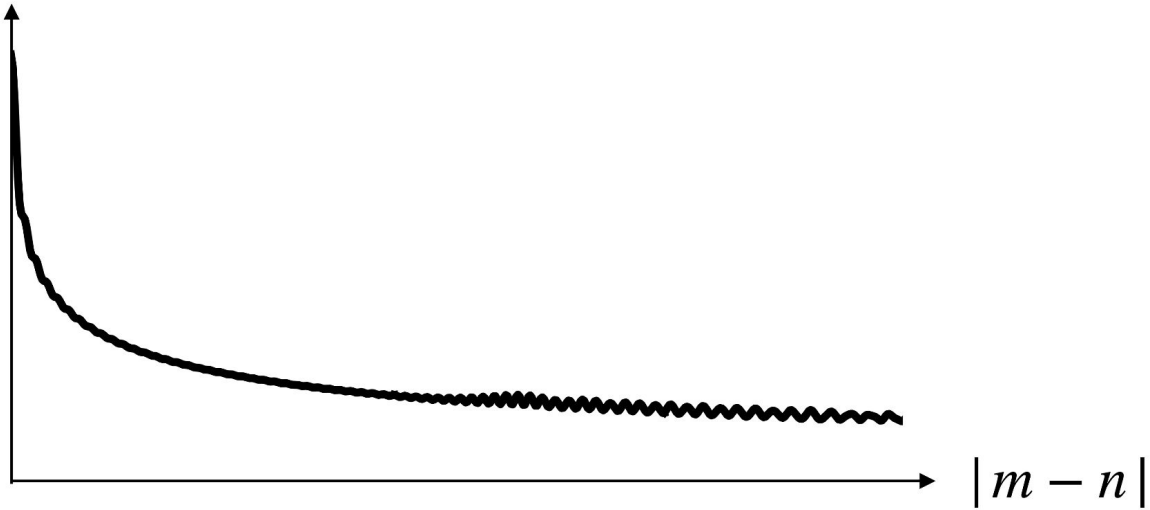
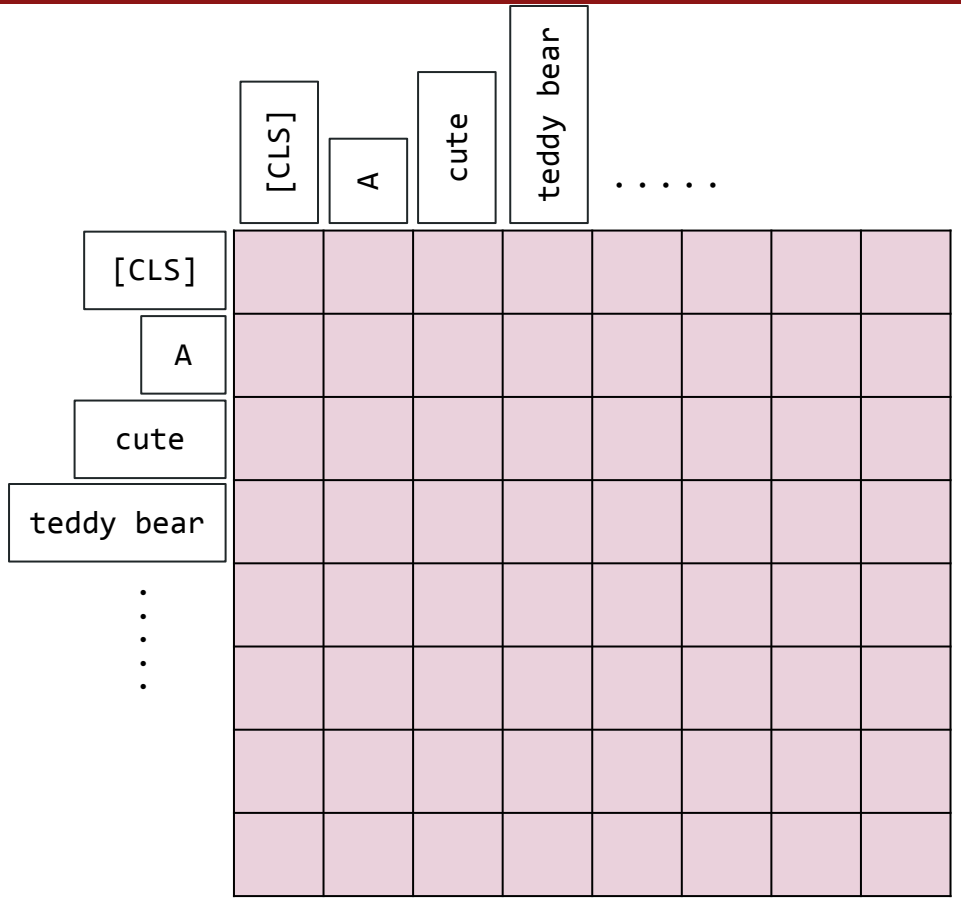


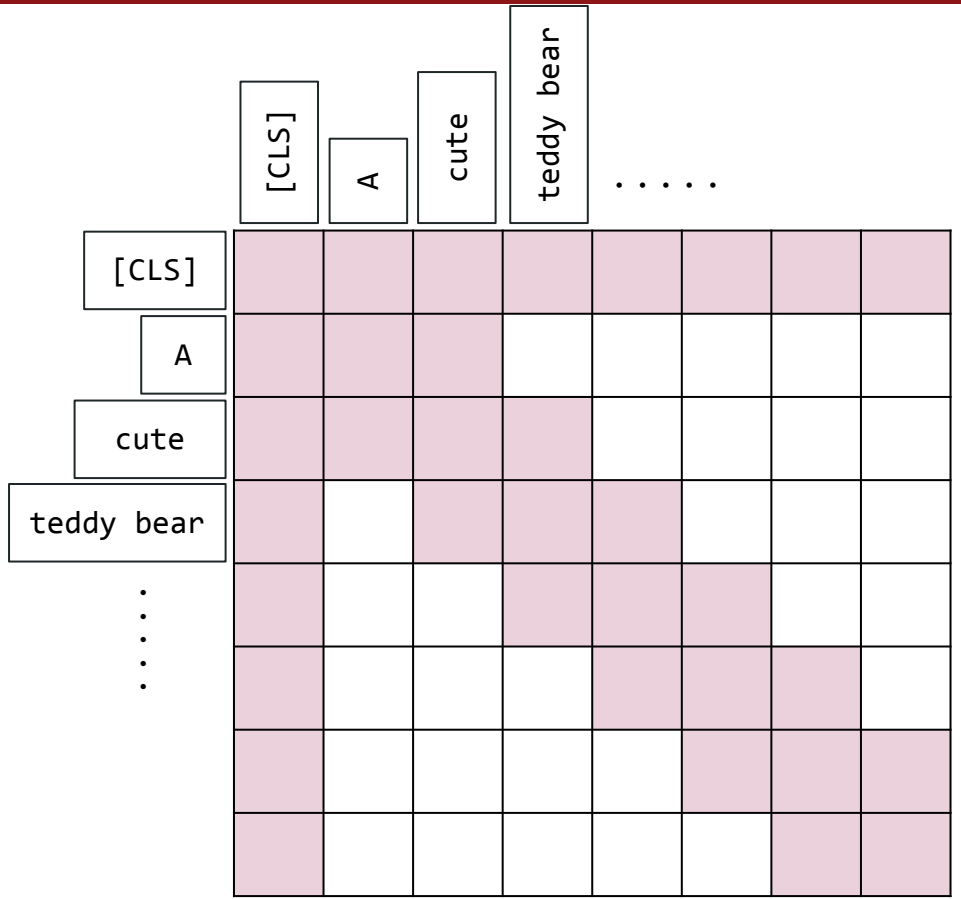
Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Sparse attention: Longformer



Longformer: The Long-document Transformer, Beltagy et al., 2020.

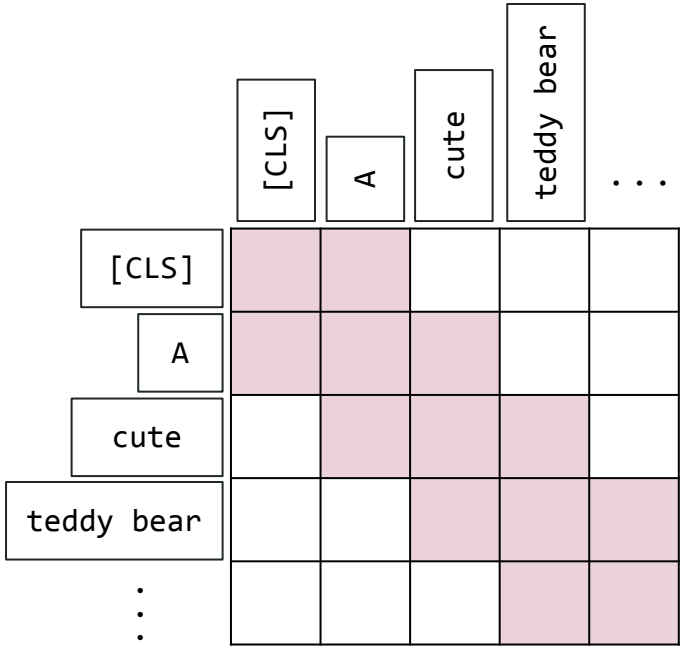
Sparse attention: Longformer



Longformer: The Long-document Transformer, Beltagy et al., 2020.

Leveraging local and global attention

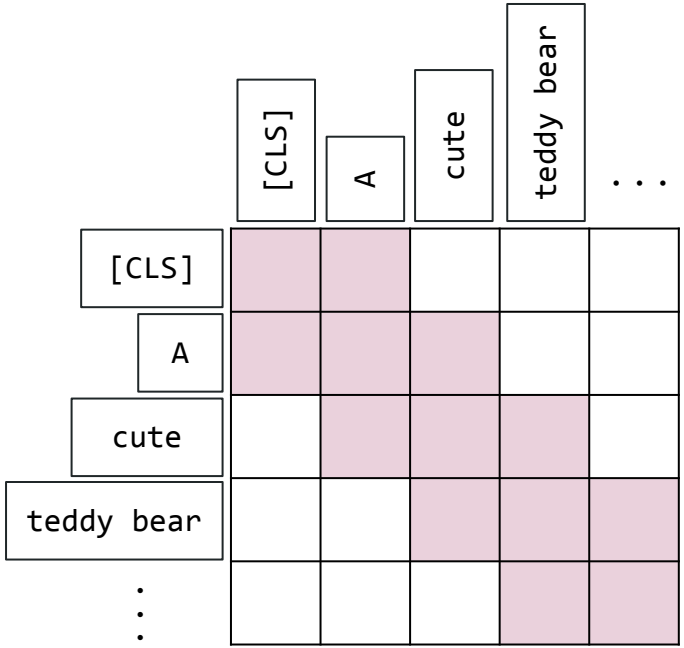
SWA = **S**liding **W**indow **A**ttention

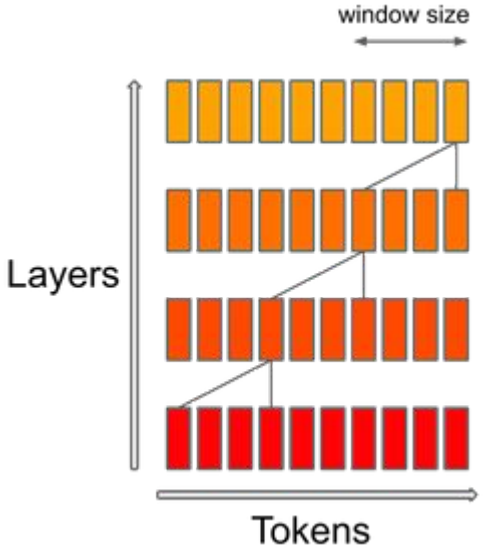


Variations include **interleaving local** and **global attention layers**

Leveraging local and global attention

SWA = **S**liding **W**indow **A**ttention



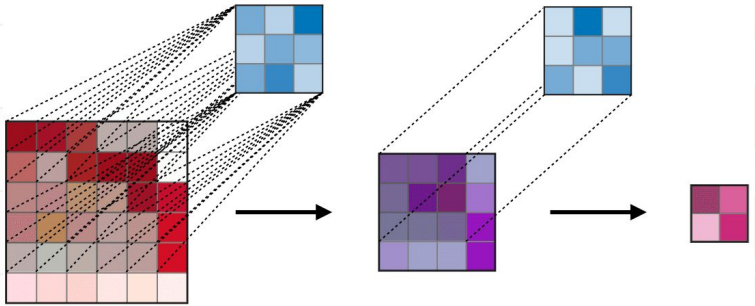


Mistral 7B

Right figure from Mistral 7B announcement, 2023.

Leveraging local and global attention

Similar to "**receptive field**" in computer vision



VIP Cheatsheet: Convolutional Neural Networks, *Amidi, 2018.*

Sharing attention heads

Idea. Share key/value attention heads within groups of queries

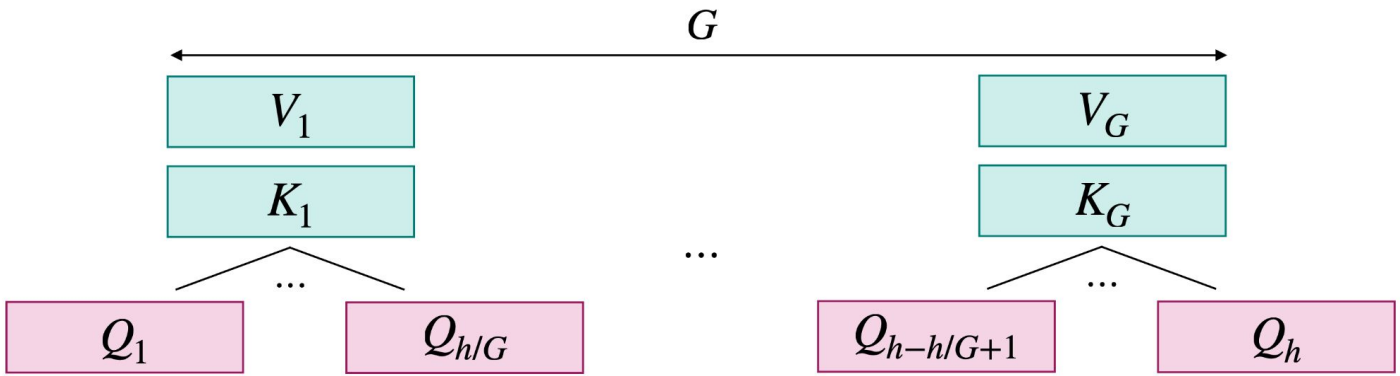


Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Sharing attention heads

$G = 1$

Multi-**Q**uery **A**ttention (MQA)

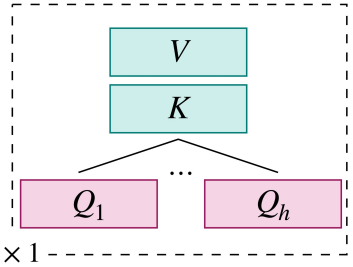


Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Sharing attention heads

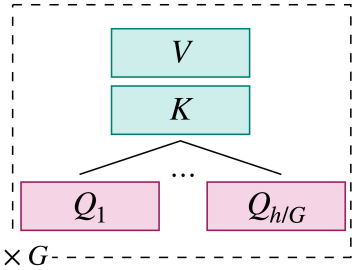
$G = 1$	**M**ulti-**Q**uery **A**ttention (MQA)	
$1 < G < h$	**G**roup-**Q**uery **A**ttention (GQA)	

Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Sharing attention heads

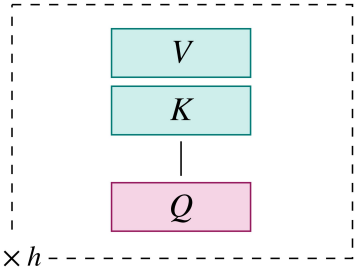
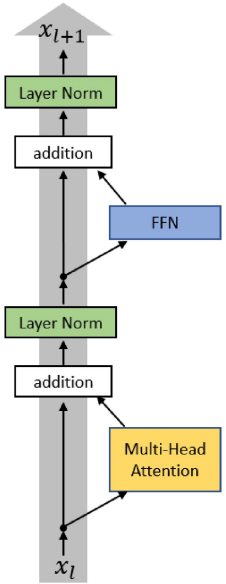
$G = 1$	**M**ulti-**Q**uery **A**ttention (MQA)	
$1 < G < h$	**G**roup-**Q**uery **A**ttention (GQA)	
$G = h$	**M**ulti-**H**ead **A**ttention (MHA)	

Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Different kinds of layer normalization

Post-Norm

$$\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$



After

residual connection

On Layer Normalization in the Transformer Architecture, Xiong et al., 2020.

Different kinds of layer normalization

Post-Norm

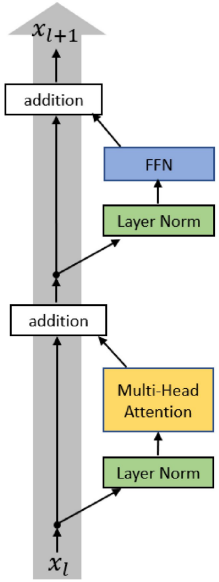
$$\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

After

residual connection

Pre-Norm

$$\text{Output} = x + \text{SubLayer}(\text{LayerNorm}(x))$$



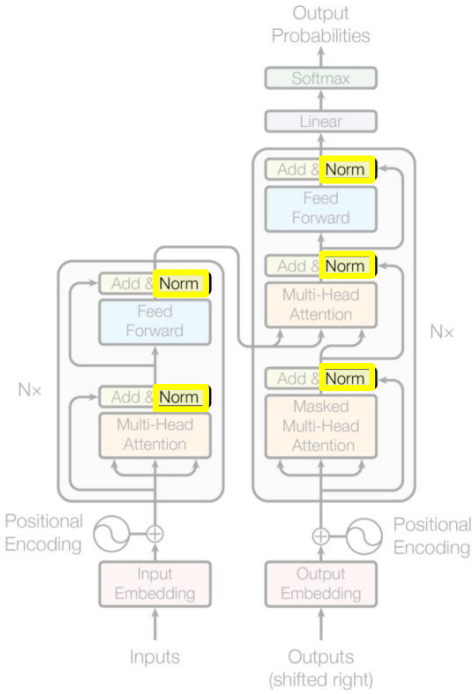
Before

residual connection

On Layer Normalization in the Transformer Architecture, Xiong et al., 2020.

Layer normalization

LN = **L**ayer **N**ormalization



$$\boxed{\mathrm{LN}(x) = \gamma\hat{x} + \beta}$$

with $\hat{x} = \dfrac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$

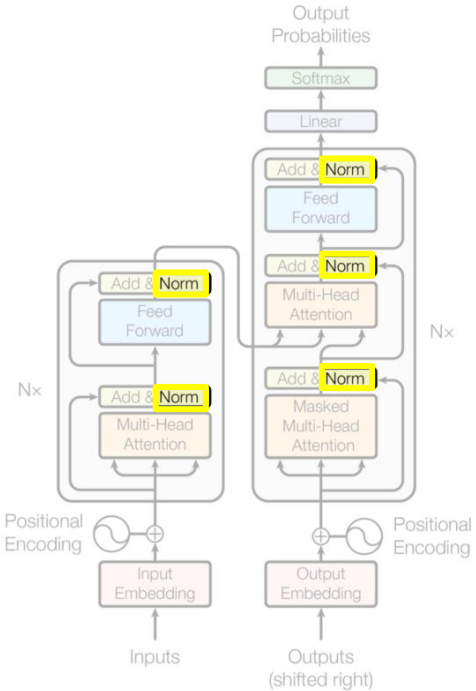
where $\mu = \dfrac{1}{d}\sum_{i=1}^{d} x_i$ and $\sigma^2 = \dfrac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$

Benefits. Helps with training stability and convergence.

Attention Is All You Need, Vaswani et al., 2017. *Layer Normalization*, Ba et al., 2016.

Normalization now!

LN = **L**ayer **N**ormalization



$$\mathrm{LN}(x) = \gamma\hat{x} + \beta$$

$$\mathrm{RMSNorm}(x) = \gamma\frac{x}{\mathrm{RMS}(x)}$$

Attention Is All You Need, Vaswani et al., 2017. *Root Mean Square Layer Normalization*, Zhang et al., 2019.

Evolution of normalization

Before	Nowadays
Post-Norm +	**Pre**-Norm +
LayerNorm	**RMS**Norm



Transformers & Large Language Models

Transformer model families

Scaling laws

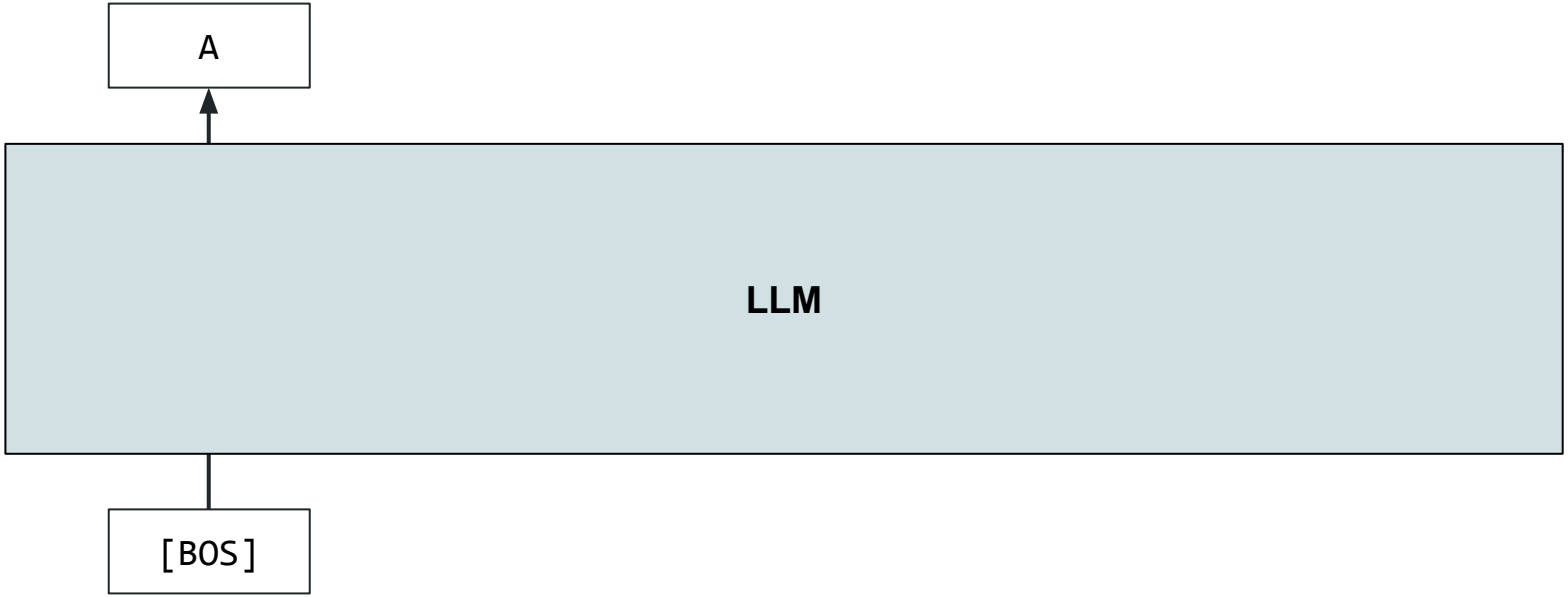
Expert routing

Architecture optimizations

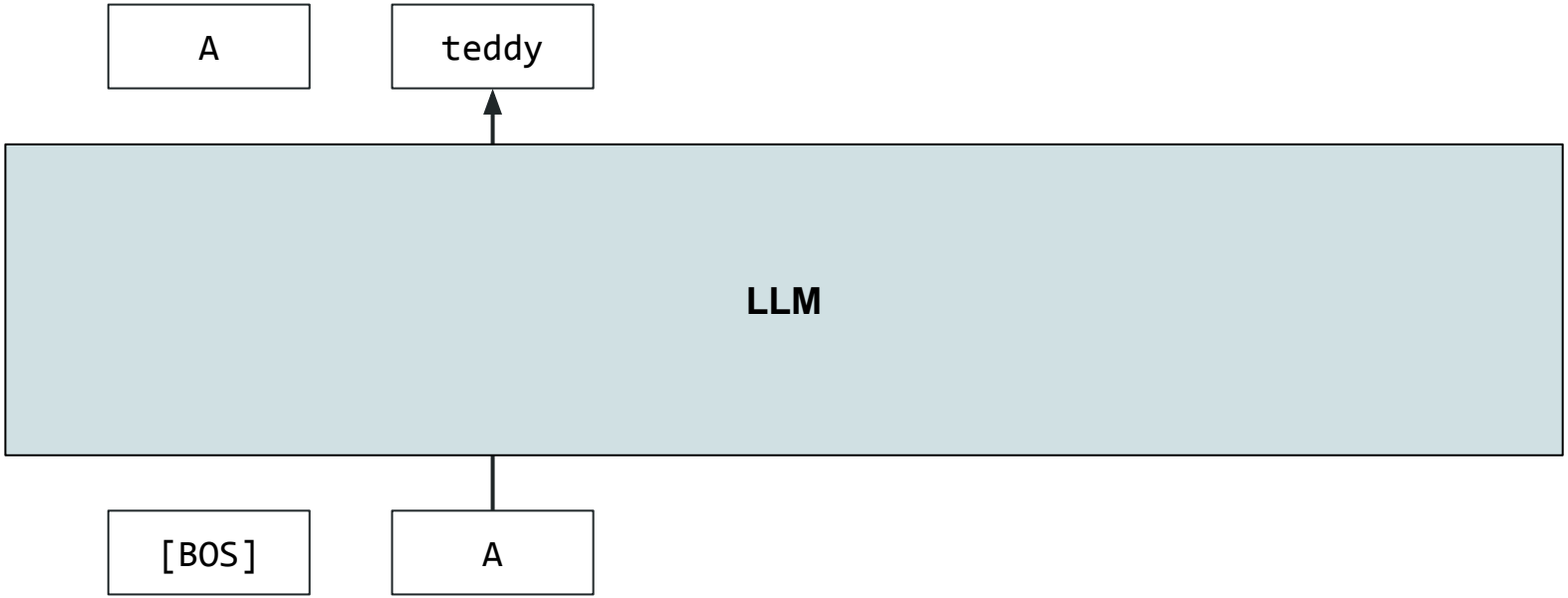
Decoding strategies

Prompting and reasoning

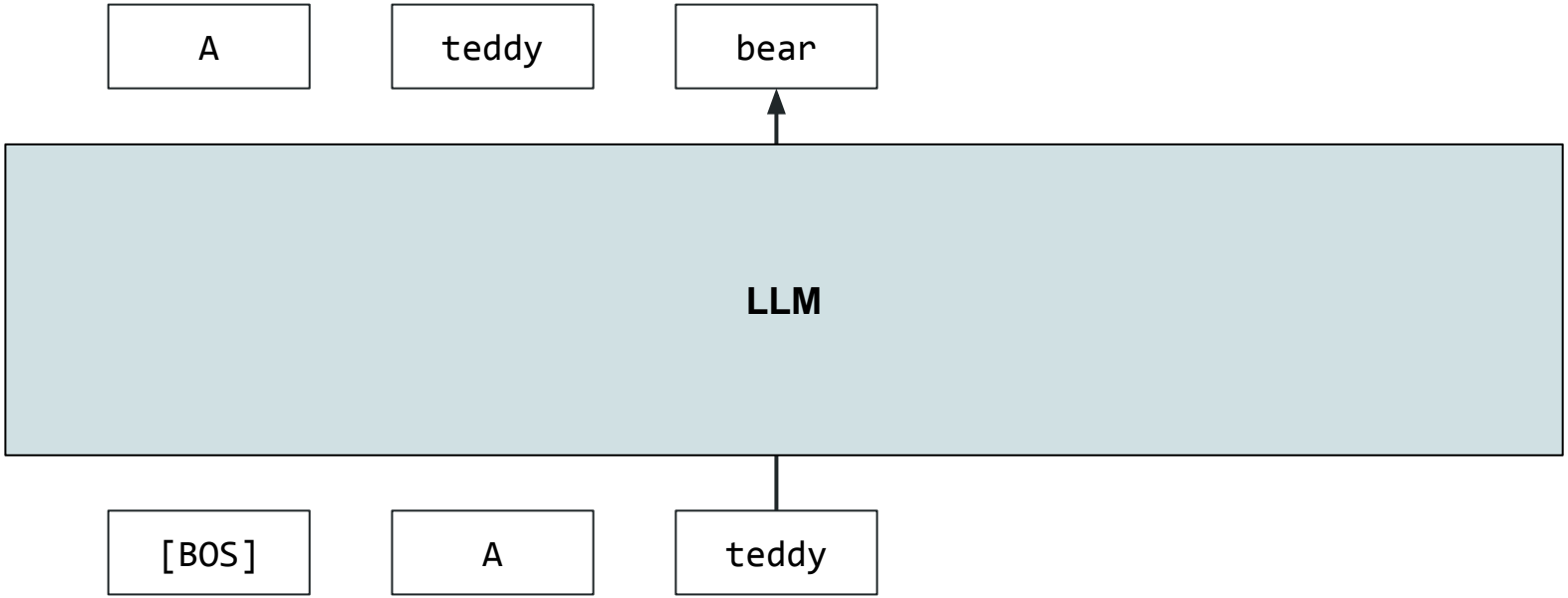
Next token prediction



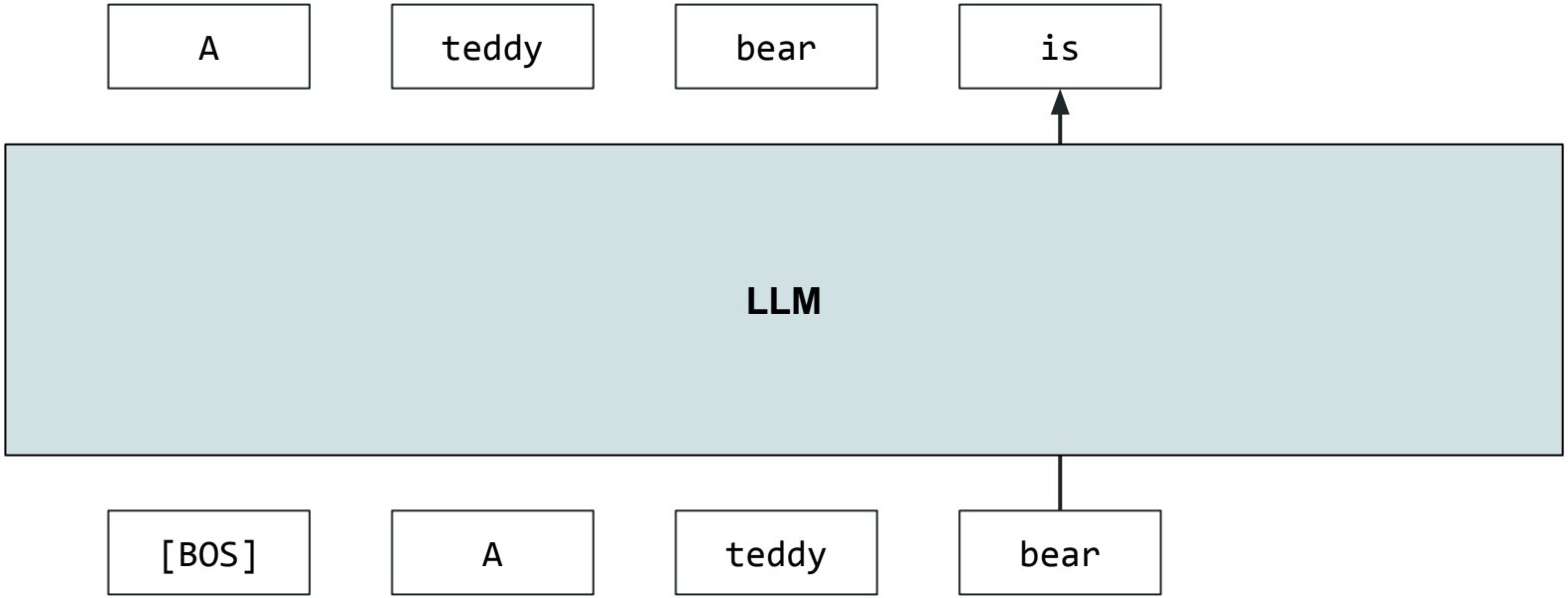
Next token prediction



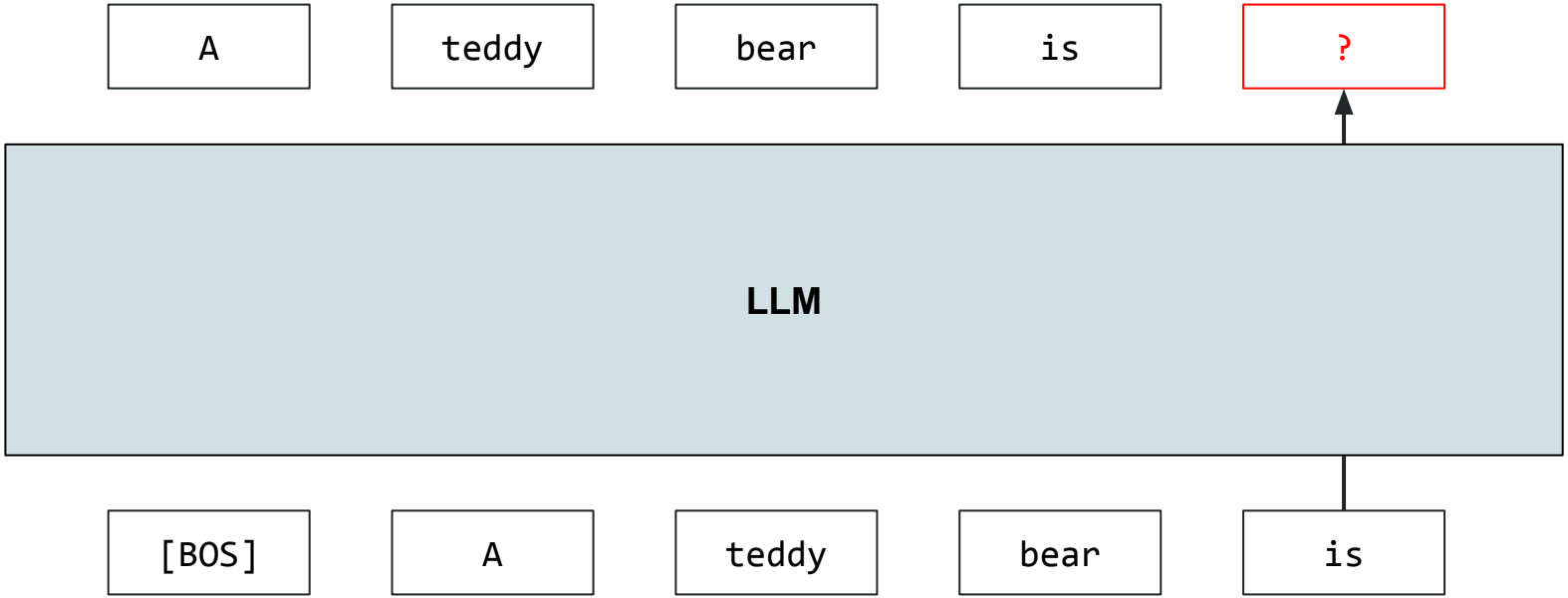
Next token prediction



Next token prediction



Next token prediction



Predicting next token

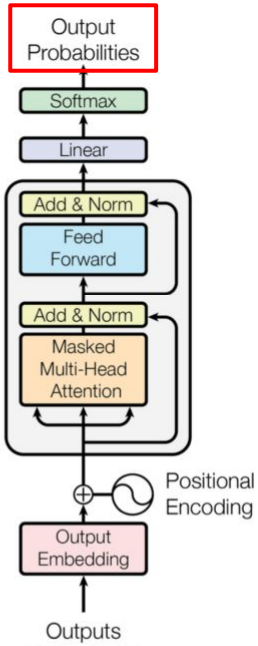
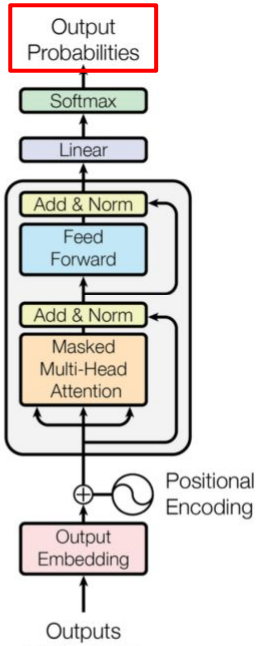


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Predicting next token



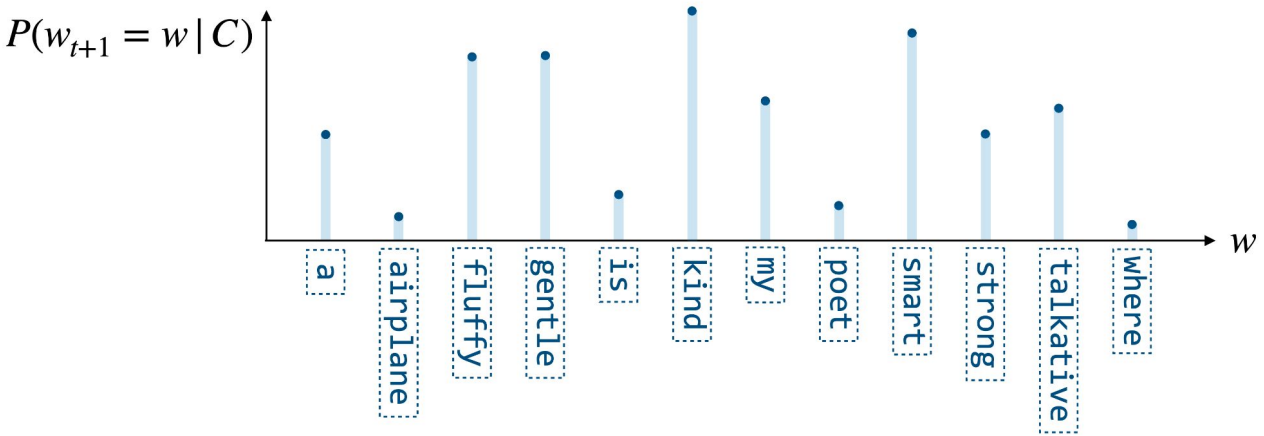
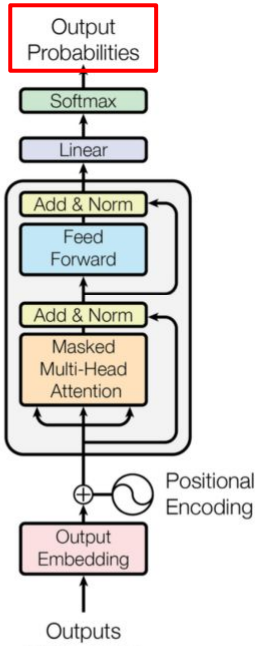
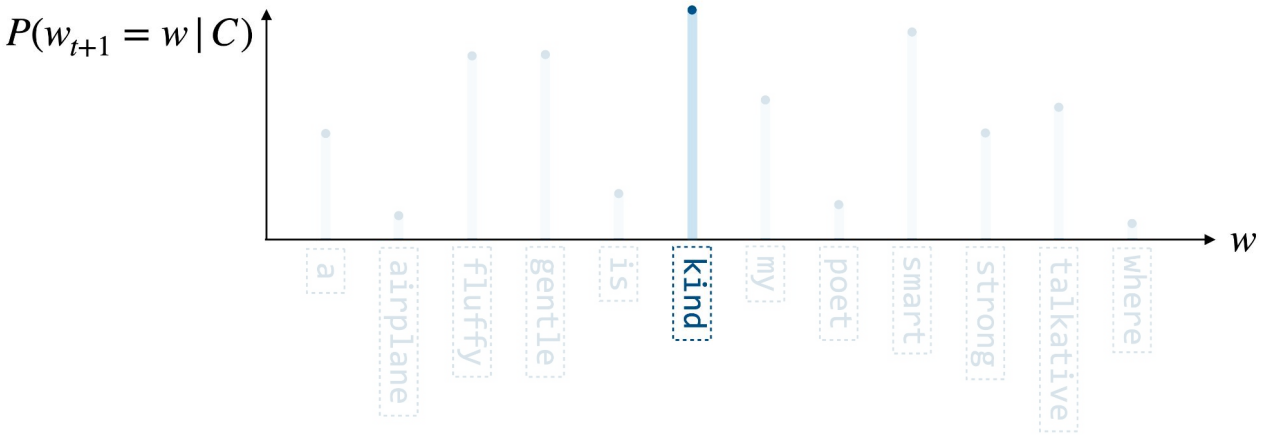


Figure adapted from Attention is All You Need, Vaswani et al., 2017. Super Study Guide: Transformers and Large Language Models, Amidi, 2024.

Predicting next token with greedy decoding



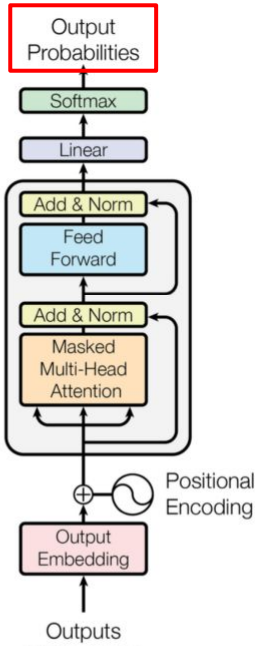
1st idea. Take token with highest predicted probability



Problem: Local optimum ≠ Global optimum

Figure adapted from Attention is All You Need, Vaswani et al., 2017. Super Study Guide: Transformers and Large Language Models, Amidi, 2024.

Predicting next token with beam search



2nd idea. Keep k paths that are the most likely

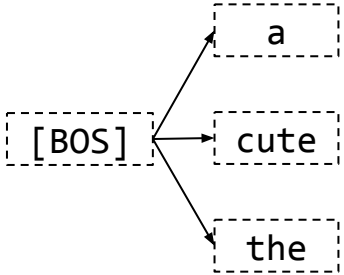
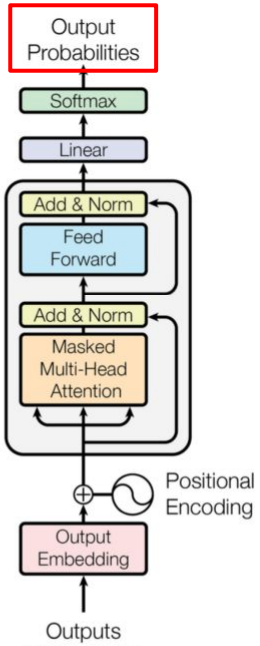


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Predicting next token with beam search



2nd idea. Keep k paths that are the most likely

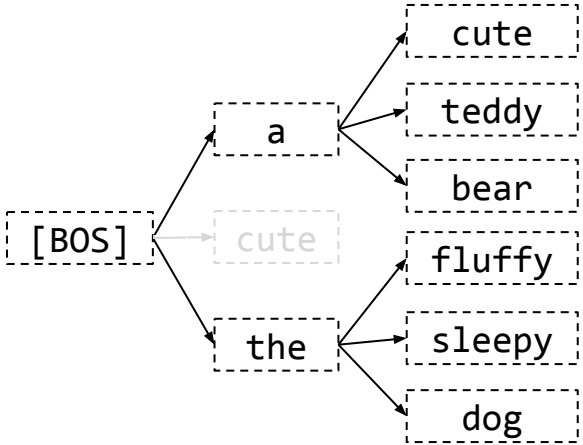
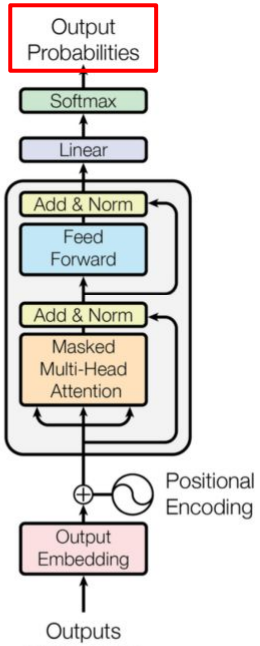


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Predicting next token with beam search



2nd idea. Keep k paths that are the most likely

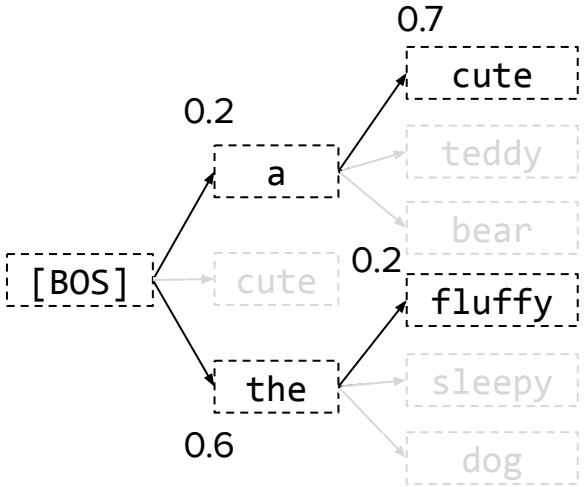
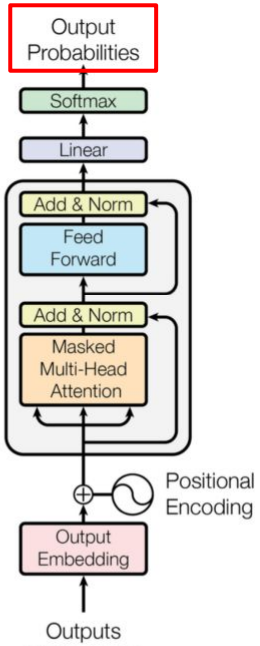


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Predicting next token with beam search



2nd idea. Keep k paths that are the most likely

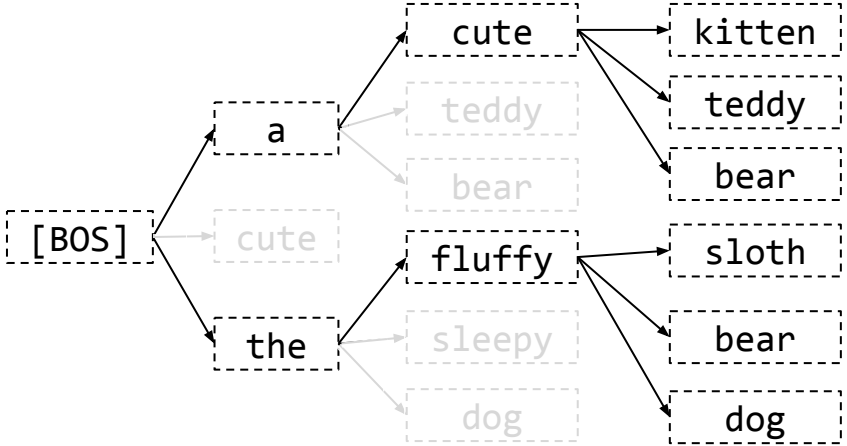
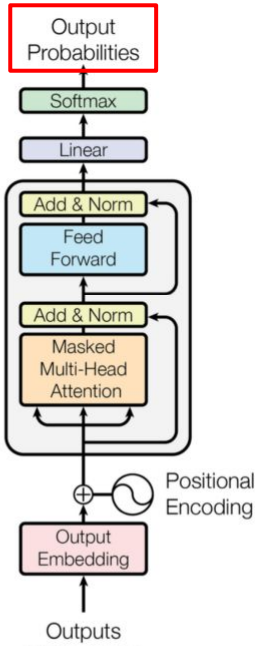
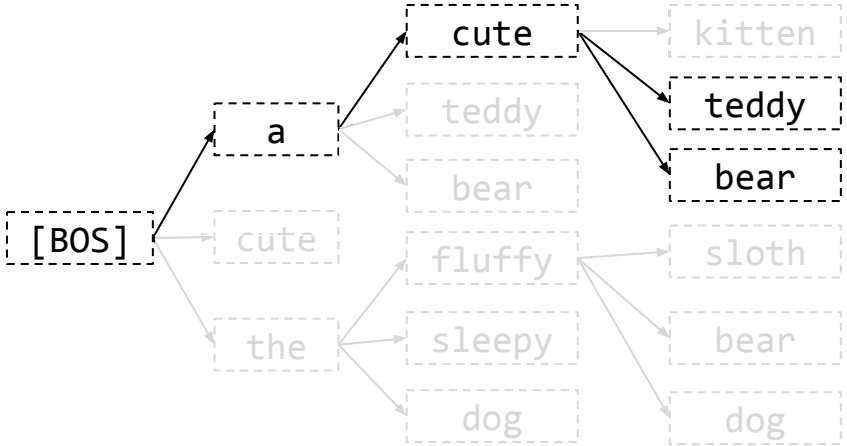


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Predicting next token with beam search



2nd idea. Keep k paths that are the most likely

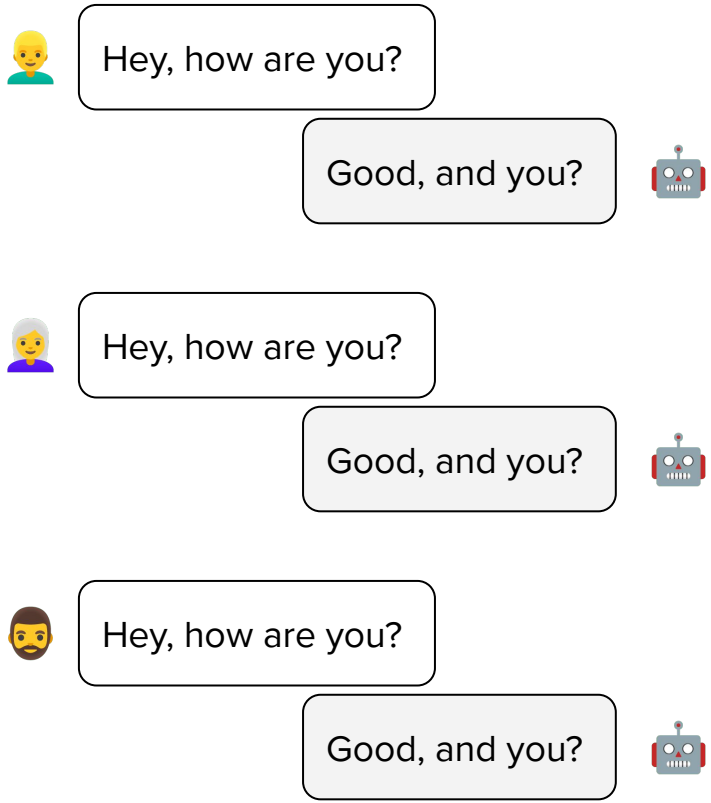


Ends at [EOS]

Problem: More computations + still lacks creativity

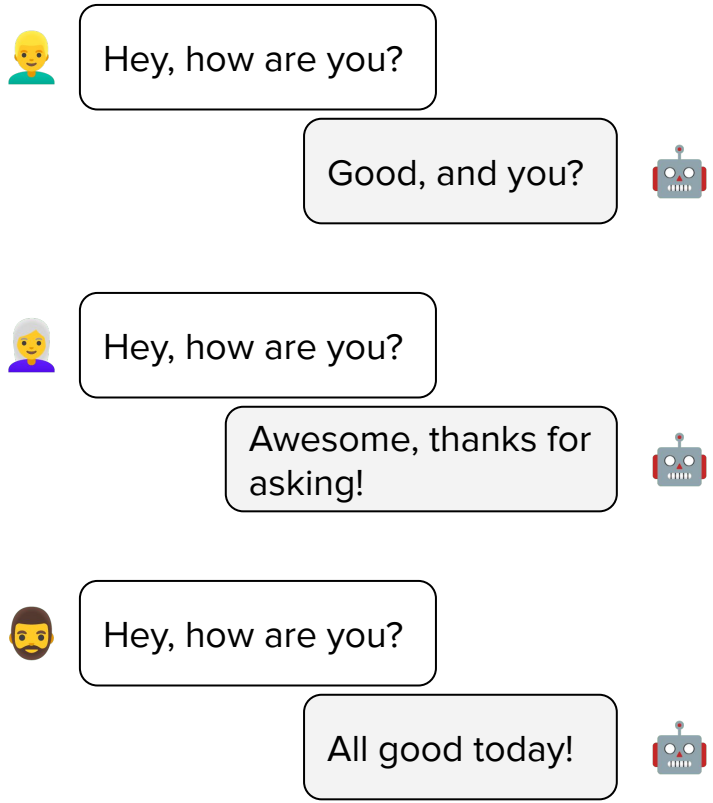
Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Problem with "deterministic" schemes

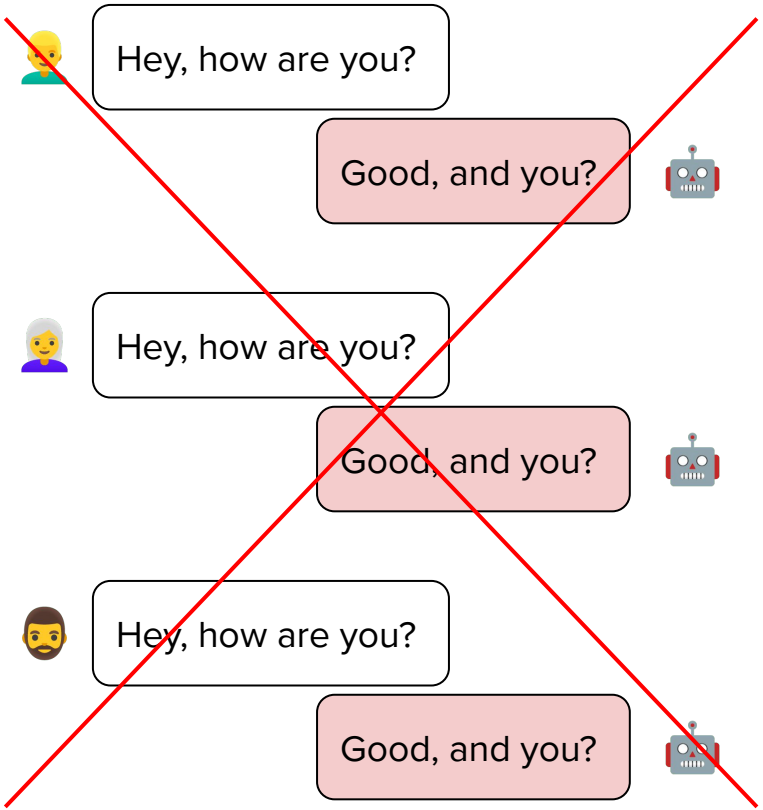


Deterministic

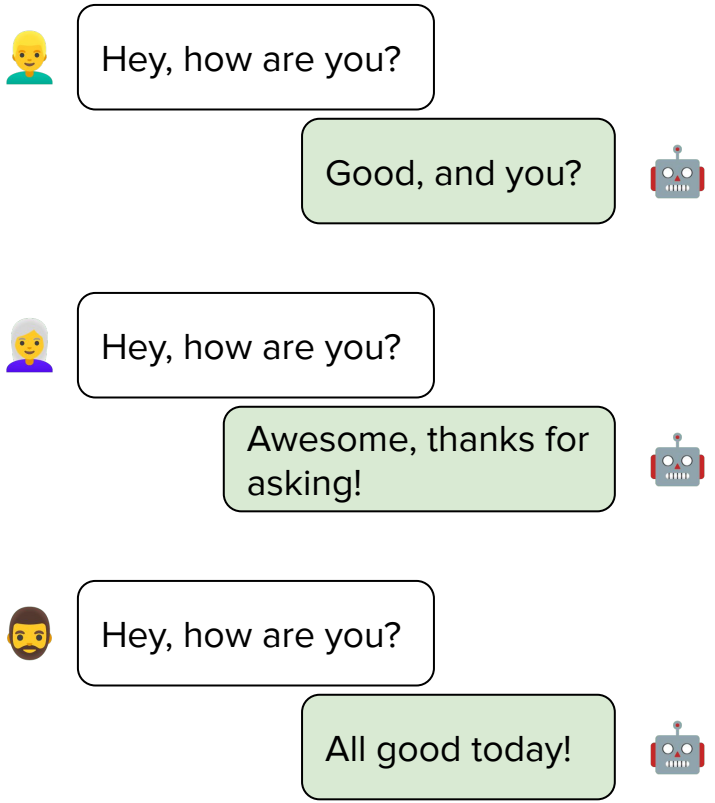
Problem with "deterministic" schemes



Problem with "deterministic" schemes

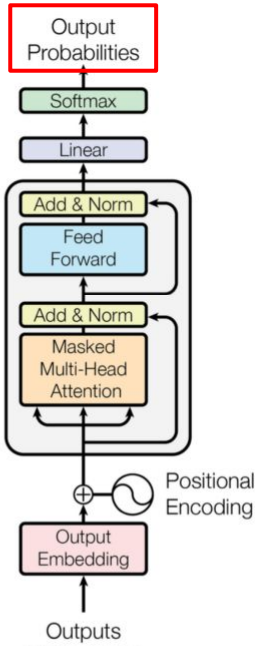


Deterministic



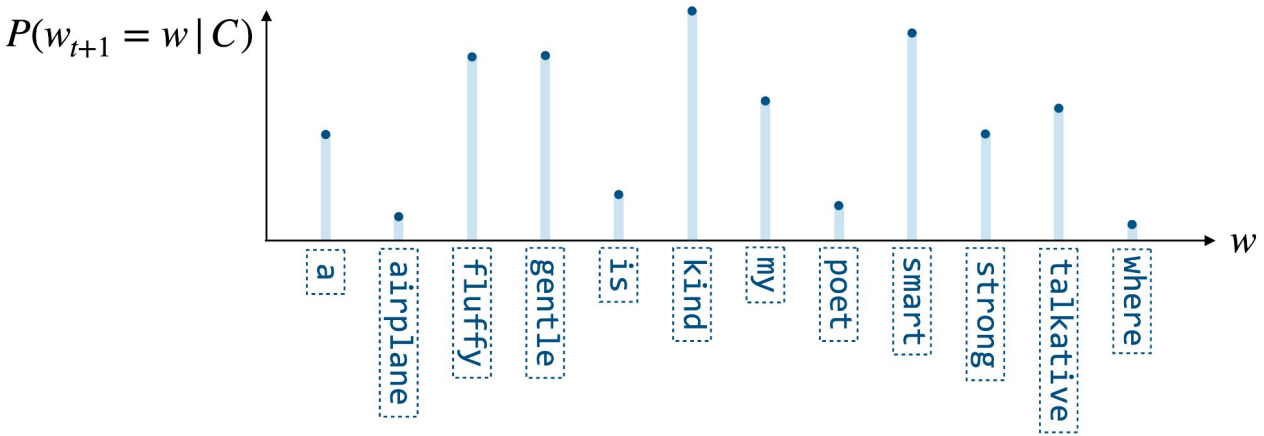
Non-deterministic

Predicting next token with sampling



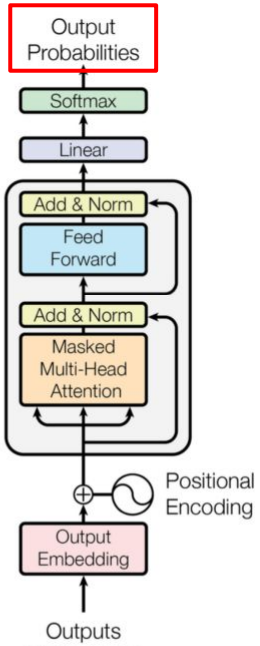
3rd idea. Sample next token from probability distribution:

$$\widehat{w}_{t+1} \sim P(w_{t+1}|C)$$



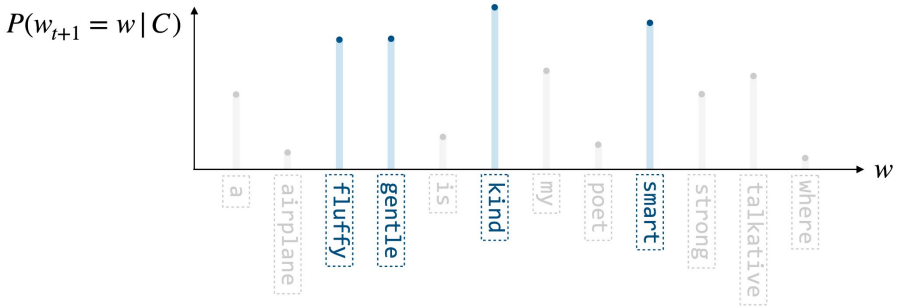
Left: adapted from Attention Is All You Need, Vaswani et al., 2017. Right: from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Sampling strategies



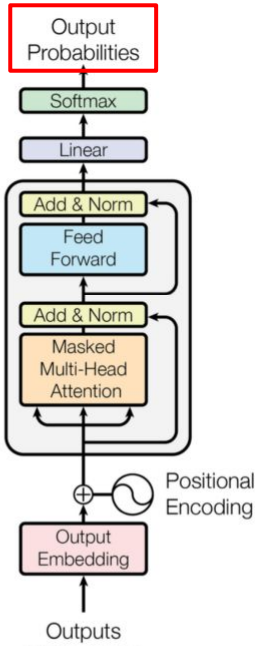
- **Top-k**: Sample among top k most probable tokens

$k = 4$



Left: adapted from Attention Is All You Need, Vaswani et al., 2017. Right: from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Sampling strategies

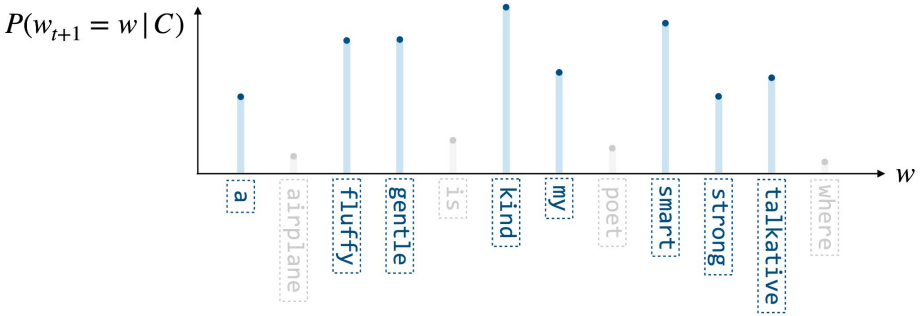


- **Top-k**: Sample among top k most probable tokens

$k = 4$

- **Top-p**: Random sample among smallest set of tokens with cumulative probability $\geqslant p$

$p = 90\%$

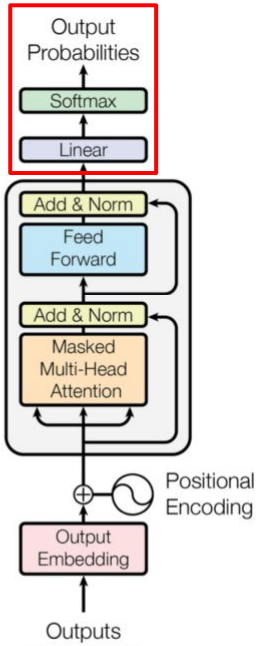


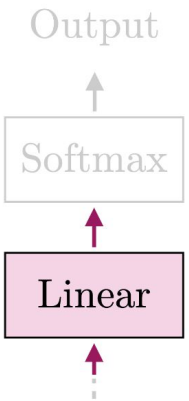
Left: adapted from Attention Is All You Need, Vaswani et al., 2017. Right: from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

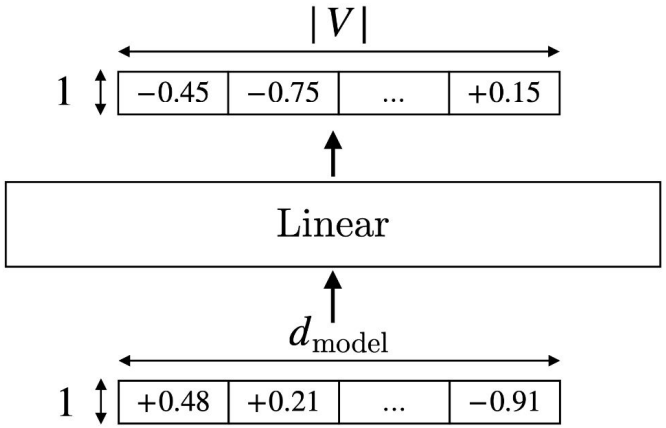
Next token prediction

But **how** are probabilities **obtained**?

Probability computation

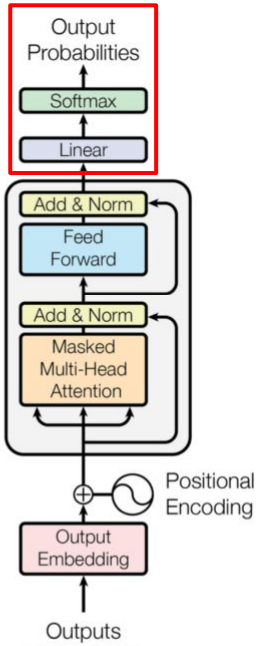


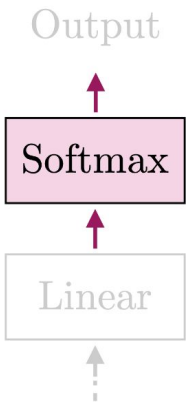


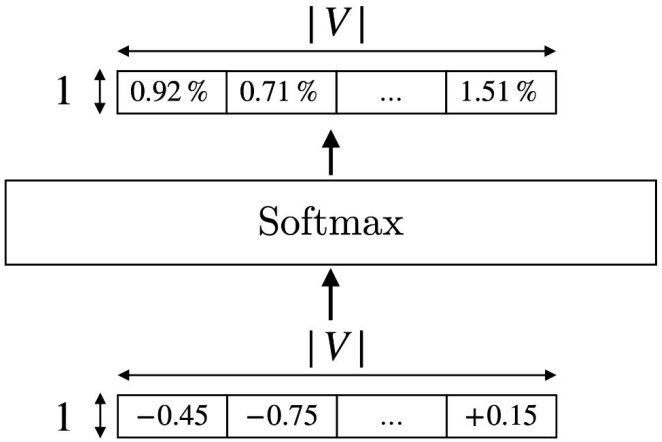


Left: adapted from Attention Is All You Need, Vaswani et al., 2017. Right: from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Probability computation

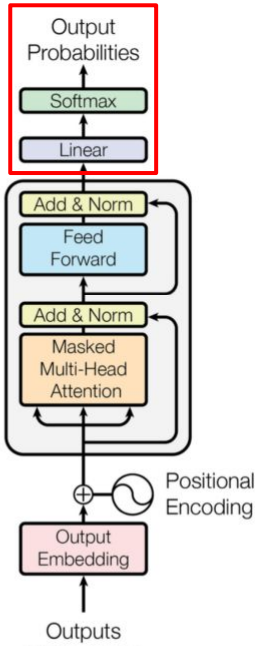


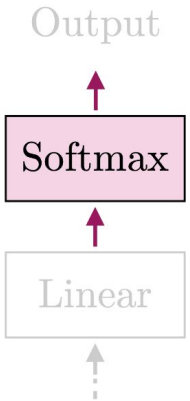




Left: adapted from Attention Is All You Need, Vaswani et al., 2017. Right: from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Temperature allows to tweak output probabilities





$$P_{\text{adj}}(w_{t+1} = w_i | C) = \frac{\exp\left(\frac{x_i}{T}\right)}{\sum_{j=1}^{n} \exp\left(\frac{x_j}{T}\right)}$$

Left: adapted from Attention Is All You Need, Vaswani et al., 2017. Right: from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

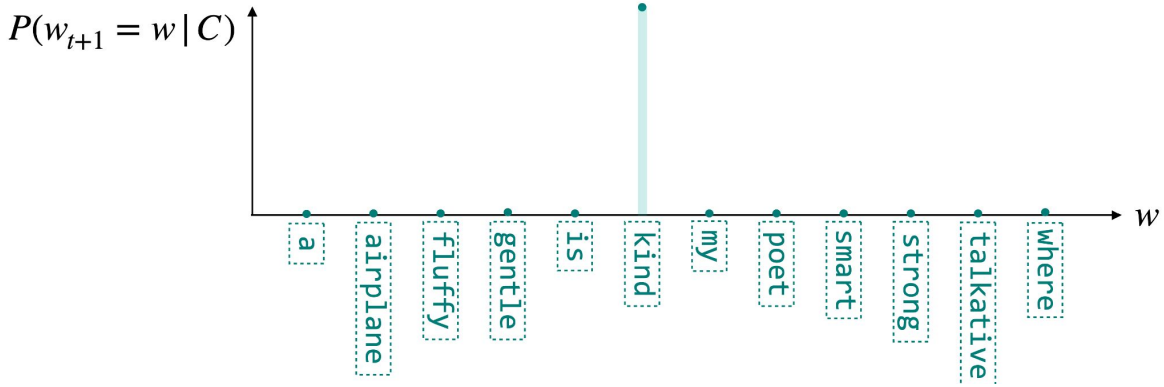
Impact of temperature on probabilities

Small T ?

High T ?

Impact of temperature on probabilities

Small T



High T

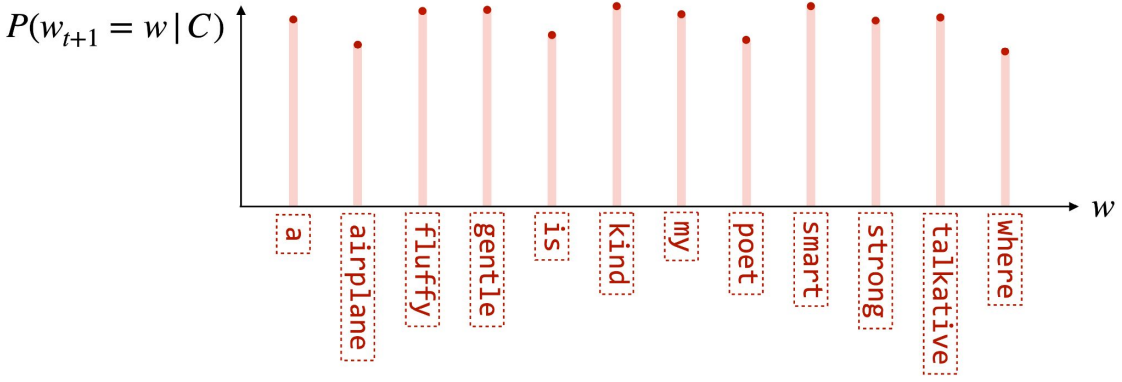
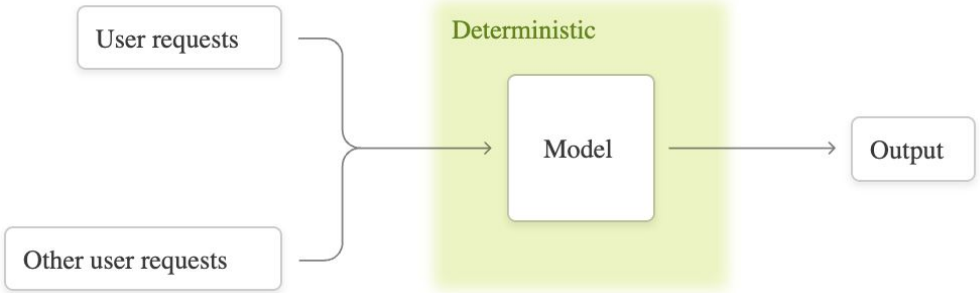
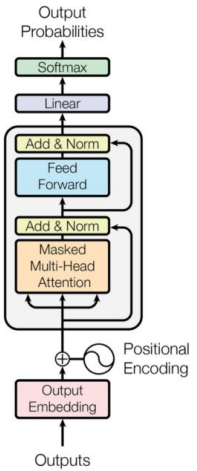


Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

LLM determinism is hard

Yes...



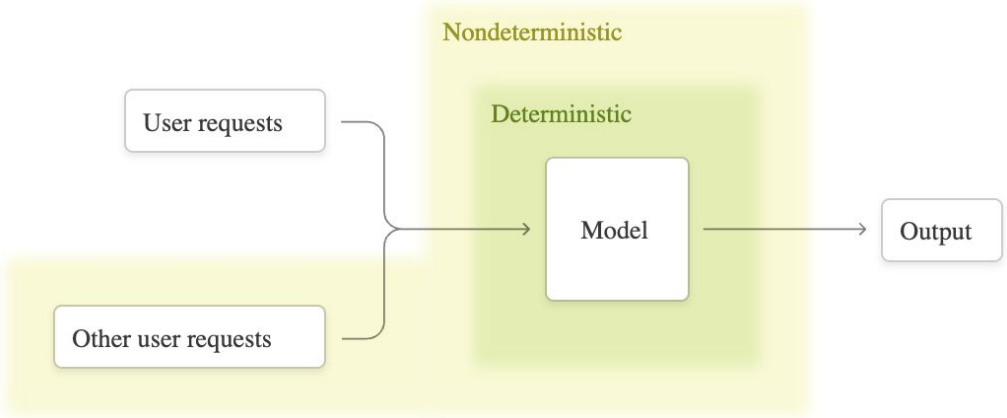


Top: figure from Defeating Nondeterminism in LLM Inference, He et al., 2025. Bottom: adapted from Attention Is All You Need, Vaswani et al., 2017.

LLM determinism is hard

Yes...

But...



Defeating Nondeterminism in LLM Inference, He et al., 2025.

Constraining the output via guided decoding

Motivation. Generate output in a specific format

Input prompt

Generate a description of my 33-year old teddy bear who likes reading.

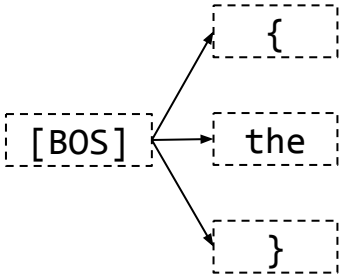
Do this in JSON format.

Desired output (JSON)

```
{
    "first_name": "teddy",
    "last_name": "bear",
    "age": 33,
    "hobby": "reading"
}
```

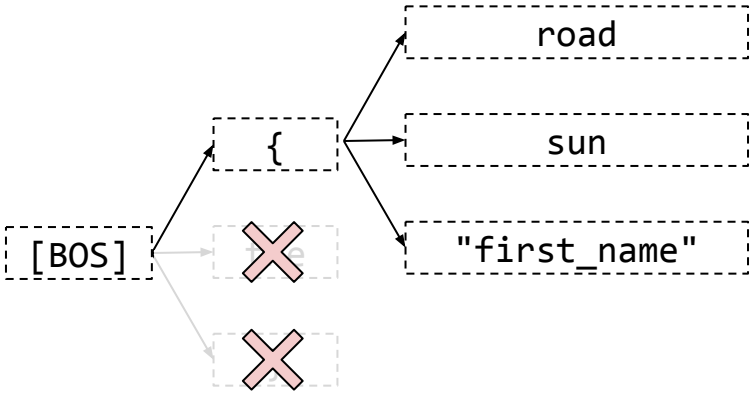
Constraining the output via guided decoding

Idea. Only allow "valid" next tokens



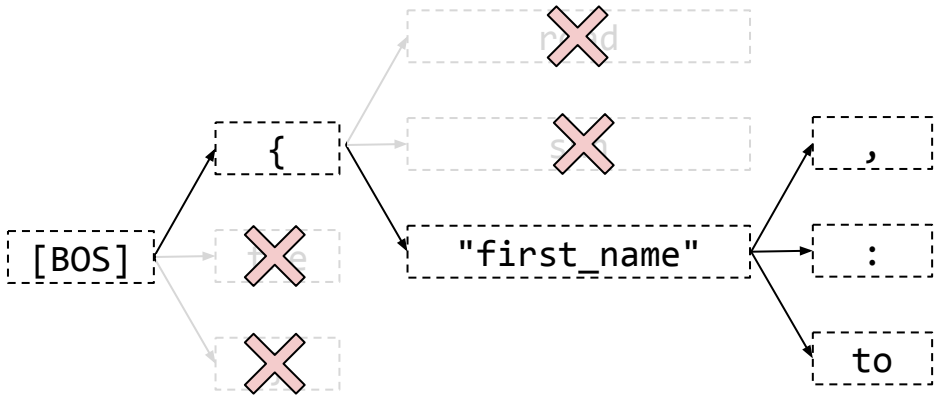
Constraining the output via guided decoding

Idea. Only allow "valid" next tokens



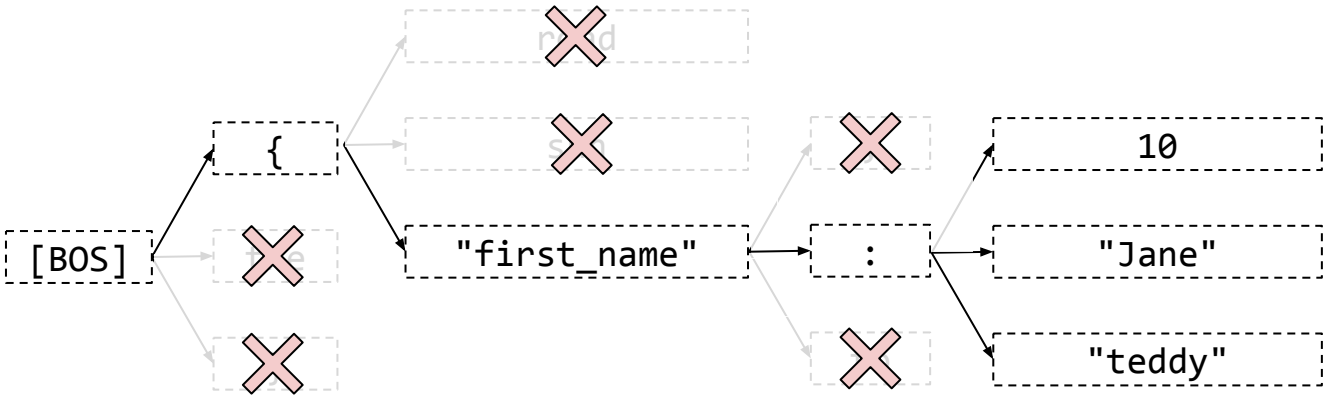
Constraining the output via guided decoding

Idea. Only allow "valid" next tokens



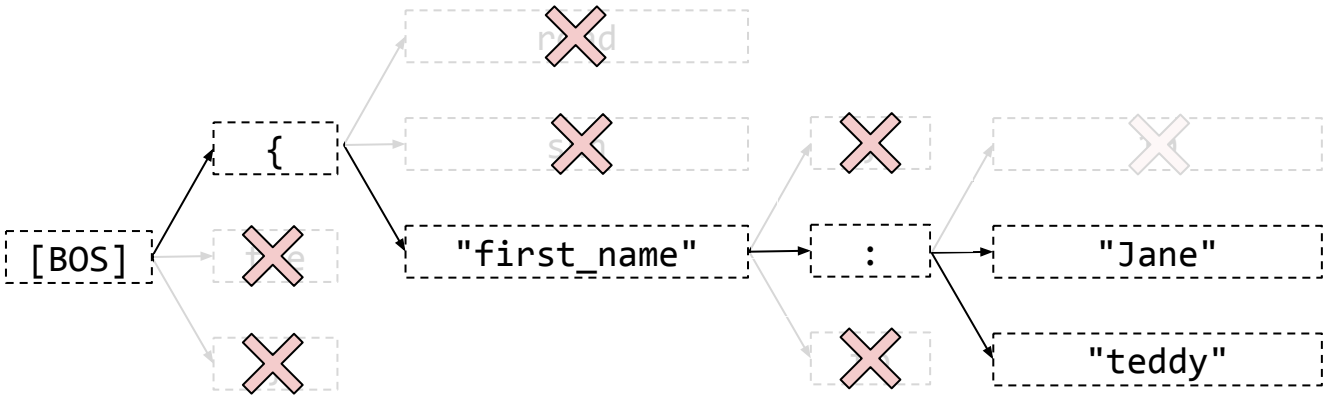
Constraining the output via guided decoding

Idea. Only allow "valid" next tokens



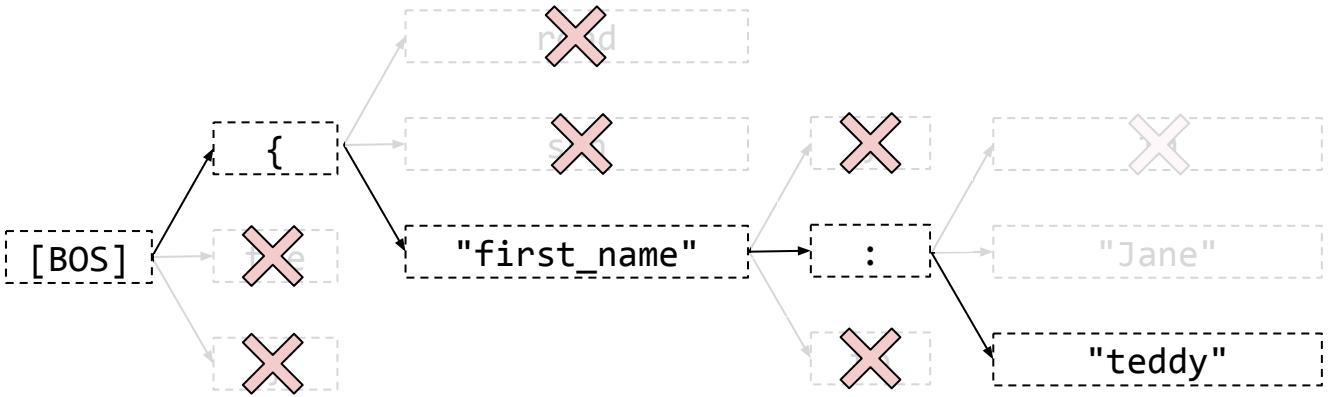
Constraining the output via guided decoding

Idea. Only allow "valid" next tokens



Constraining the output via guided decoding

Idea. Only allow "valid" next tokens





Transformers & Large Language Models

Transformer model families

Scaling laws

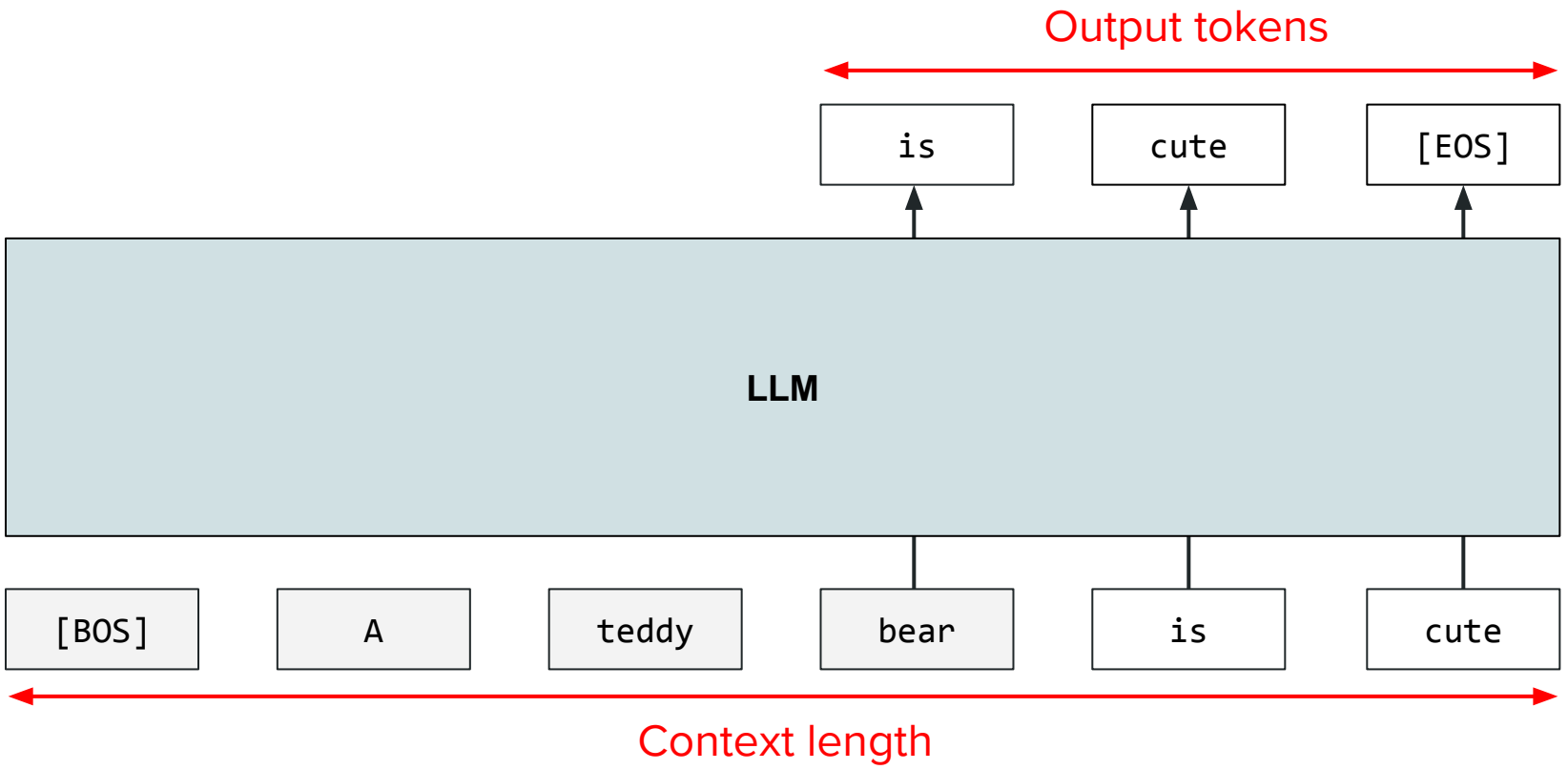
Expert routing

Architecture optimizations

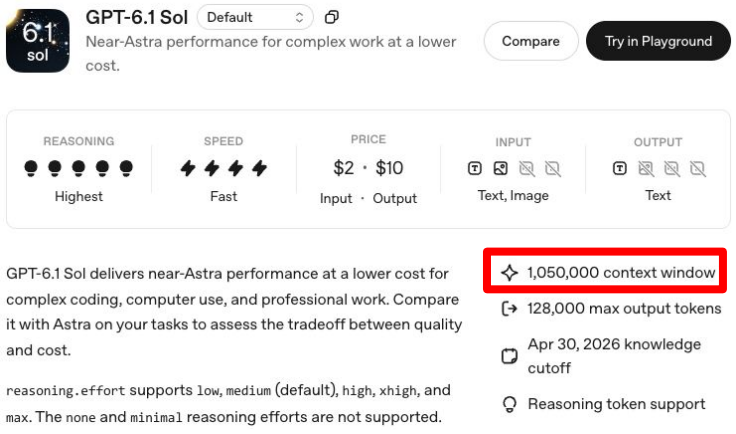
Decoding strategies

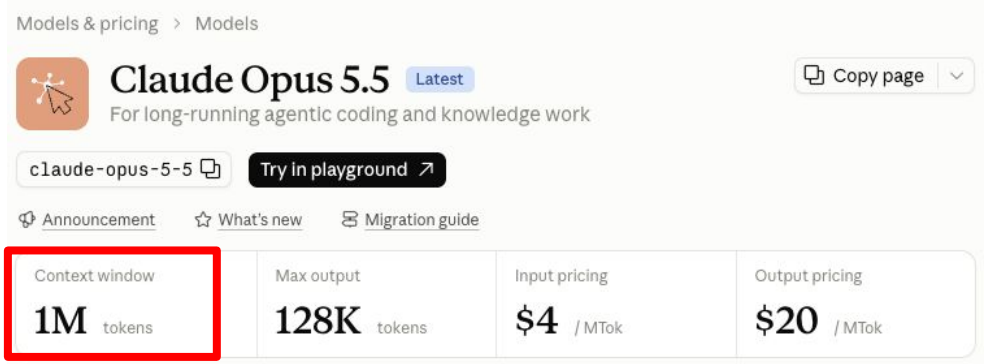
Prompting and reasoning

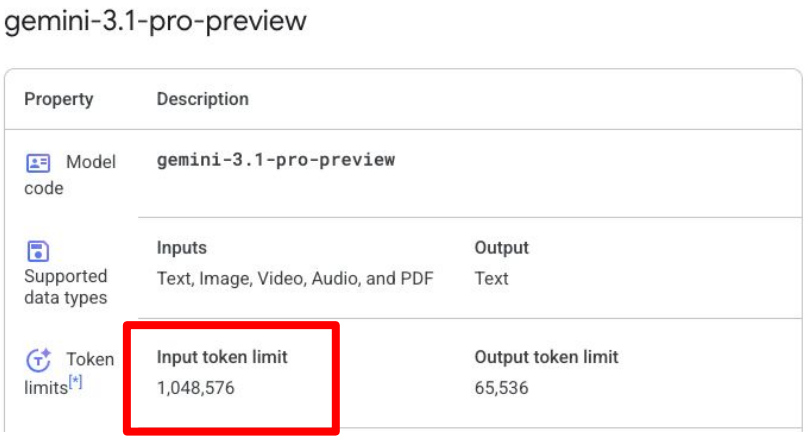
Terminology



Orders of magnitude



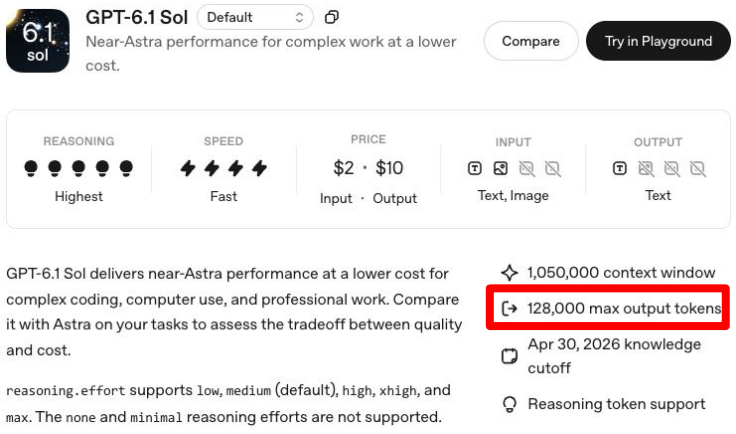




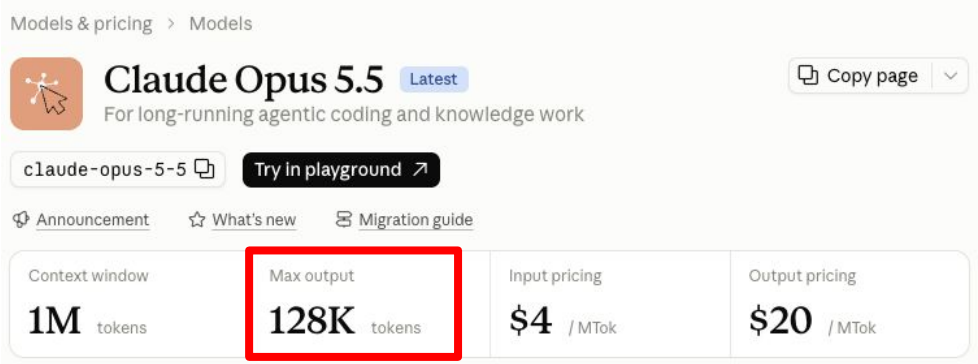
Context length: O(1M) tokens

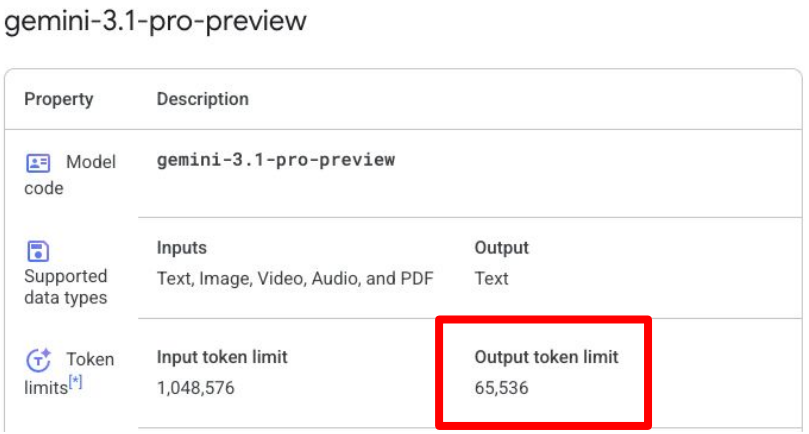
Screenshots taken from the API pages of OpenAI, Anthropic and Google on September 29, 2026.

Orders of magnitude



Output tokens: O(100k)





Screenshots taken from the API pages of OpenAI, Anthropic and Google on September 29, 2026.

Orders of magnitude



To support Gemini 4 Argon's capabilities across longer, more complex use cases, we are significantly expanding the model's output token limit to an industry-leading 1M tokens, up from the previous 64K tokens. When the model has the headroom to think deeply and generate hundreds of thousands of tokens in a single trajectory, it adds a new level of depth in reasoning to solve tough problems in one go.

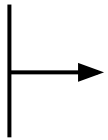
Output tokens: ~~O(100k)~~ **soon O(1M)?**

Gemini 4 Argon: our next era of frontier intelligence, *News from Google, September 30, 2026.*

Discussion

Mental model.

- 1 tokens ≃ 0.75 words
- 1 U.S. letter page with 12pt font text ≃ 400 words



1 page ≃ 500 tokens

Discussion

Mental model.

- 1 tokens ≃ 0.75 words
- 1 U.S. letter page with 12pt font text ≃ 400 words

→ 1 page ≃ 500 tokens

Orders of magnitude. What can we do with all these tokens?

- 10,000 tokens: technical report
- 100,000 tokens: book
- 1,000,000 tokens: encyclopedia

Discussion

Mental model.

- 1 tokens ≃ 0.75 words
- 1 U.S. letter page with 12pt font text ≃ 400 words

→ 1 page ≃ 500 tokens

Orders of magnitude. What can we do with all these tokens?

- 10,000 tokens: technical report
- 100,000 tokens: book
- 1,000,000 tokens: encyclopedia

Are more tokens always better?

Beware of "context rot"!

Context Rot: How Increasing Input Tokens Impacts LLM Performance, Hong et al., 2025.

Prompting: risk of "context rot"

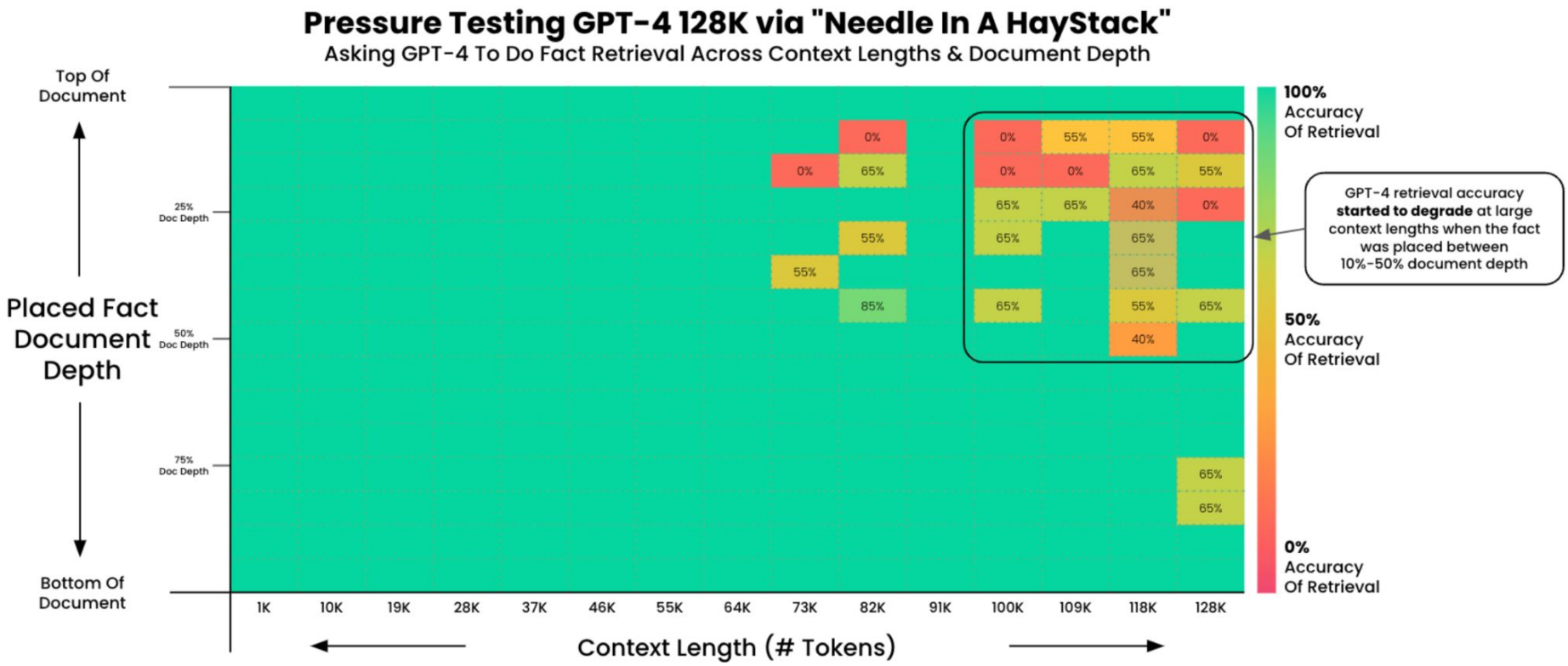


Figure from Needle in a Haystack - Pressure Testing LLMs tests, Kamradt, GitHub, 2023.

In-context learning

ICL = **I**n-**C**ontext **L**earning

Language Models are Few-Shot Learners, Brown et al., 2020.

In-context learning

ICL = **I**n-**C**ontext **L**earning

Zero-shot learning

```
How old will bear A be
next year?
```

Few-shot learning

```
Q: How old is bear A?
A: 4.

Q: How old is bear B?
A: 3.

How old will bear A be
next year?
```


Language Models are Few-Shot Learners, Brown et al., 2020.

In-context learning

ICL = **I**n-**C**ontext **L**earning

Zero-shot learning

```
How old will bear A be
next year?
```

Few-shot learning

```
Q: How old is bear A?
A: 4.

Q: How old is bear B?
A: 3.

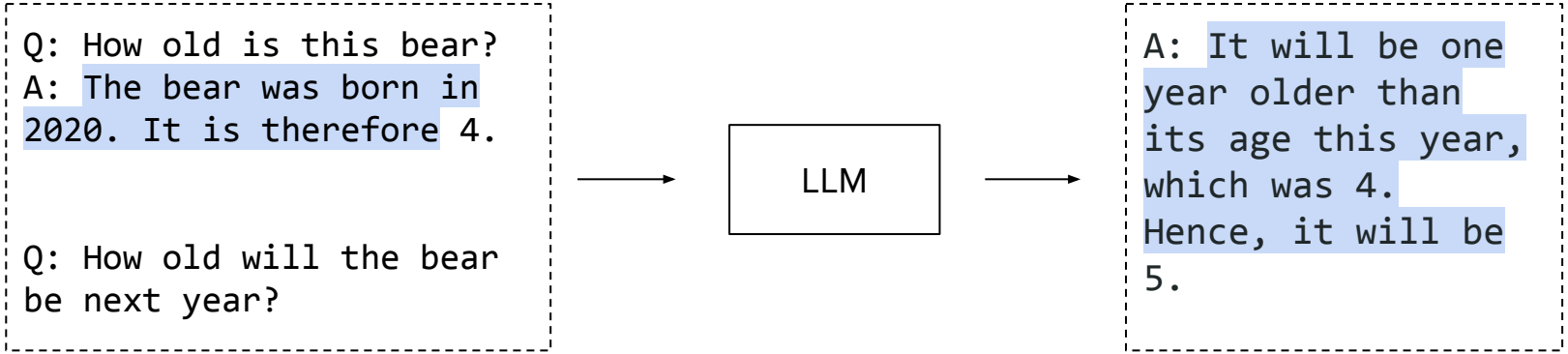
How old will bear A be
next year?
```

Discussion. *Generally* better but:

- Requires effort
- Increases computational complexity & cost
- Increases latency

Chain of thought

Idea. Explaining reasoning helps in improving performance.



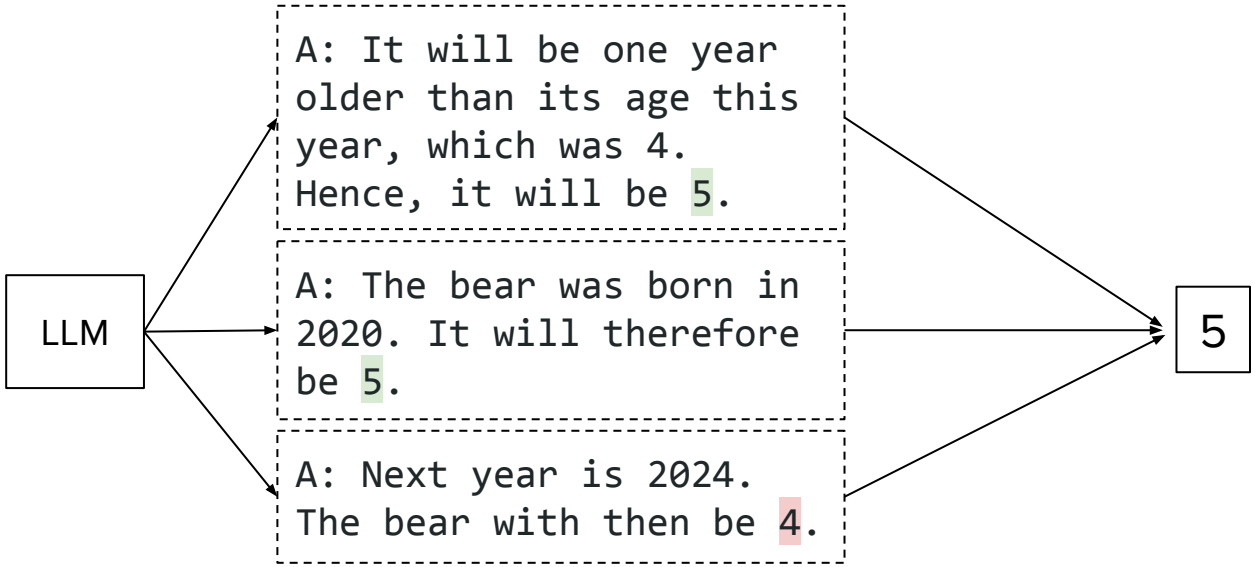
Discussion.

- Interpretability + explanation
- More tokens: higher cost and latency

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, Wei et al., 2022.

Self-consistency

Idea. Aggregating over reasoning paths improves performance.



Discussion. Trade-off between performance and added cost.

Self-Consistency Improves Chain of Thought Reasoning in Language Models, Wang et al., 2022.

Thank you for your attention!